Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.DOI

Playable but Private: In-Encoder Selective Encryption with Ascon-AEAD128 and Attack-Adaptive Protection for MJPEG Video on an FPGA SoC

ARAVIND R[1], SUHAS V J[1], RISHAB H R[1], SAMARTH B N[1], VIJAYSURYA B S[1], and CHETANA G[1] [TODO: order, IEEE membership]

[1]Department of Electronics and Communication Engineering, RV College of Engineering, Bengaluru 560059, India [TODO: e-mails]

Corresponding author: [TODO: name] (e-mail: [TODO: address]).

[TODO: funding statement, or "This work received no external funding."] **Draft, October 20, 2026. Not for distribution.**

ABSTRACT Small cameras stream over networks they do not control, yet encrypting the whole stream makes it unplayable without the key and spends cipher work on every byte. We present a JPEG/MJPEG encoder for a Zynq-7020 FPGA SoC that encrypts the sign and amplitude bits of selected DCT coefficients inside the entropy coder, with a keystream from the NIST SP 800-232 standard Ascon-AEAD128. We prove that the transformation never changes a Huffman code length, so every protection level is a valid JPEG, and a 128-bit tag over the frame metadata and the coded scan rejects any modified, replayed or forged frame. The strongest level hides block positions with a keyed, per-frame permutation of restart segments that needs no frame buffer: the fabric codes and authenticates one segment per macroblock and the processor only reorders them. The design is bit-exact with a golden model in simulation and on a PYNQ-Z2 board in all six levels, adds no cycles for encryption, closes timing at 100 MHz and reaches 176 Mpixel/s with four lanes. In a benchmark of four keyless attacks on 9 sources, four re-implemented published schemes leak 0.58–0.94 edge correlation while our permutation level stays at or below 0.02, also against jigsaw solvers, sign-recovery optimization and trained neural attackers. An authenticated feedback loop with a forward-secure ratchet mitigates all 11 modelled network attacks in 0.91 s on average with no false alarms.

INDEX TERMS selective encryption, MJPEG, Ascon, lightweight cryptography, FPGA, authenticated encryption, hardware security, video surveillance

I. INTRODUCTION

Cameras on doors, corridors, production lines and drones now send video over Wi-Fi and shared Ethernet as a matter of course. Two properties matter for such a link. Confidentiality keeps a bystander on the network from watching the scene; integrity lets the viewer trust that what it shows was captured by the camera and was not edited, replayed or injected on the way. The usual answer, encrypting the transport with TLS, SRTP [1] or a VPN, provides both, but it hides the stream from every tool that is not part of the key infrastructure, and the cipher work grows with the full bitrate.

Selective encryption takes a different route. It encrypts only the parts of the compressed representation that carry most of the visual information and leaves the format intact, so standard decoders still parse the result [2]. For baseline JPEG, the sign and the appended amplitude bits of the quantized DCT coefficients can be encrypted without changing any Huffman code length, so the bitstream stays compliant and its size does not grow [3], [4]. The price is that such schemes are easy to weaken: the positions of the non-zero coefficients stay visible, and an attacker who guesses the hidden values can still see the outline of the scene [2], [5], [6]. Most published schemes also use fixed key material, so one frame the attacker knows exposes every later frame, and almost none authenticate what they encrypt.

This work starts from a practical question: how much of a secure camera link can be put into the encoder hardware itself, and what does an eavesdropper still learn? Answering

it needs three things that are usually studied apart. The hardware must encrypt at line rate without a frame buffer and without slowing the encoder. The cryptography must be standard, authenticate every frame together with its metadata, and never reuse a nonce across frames, rekeys or reboots. And the security claim must be measured the way an attacker would attack: with replacement attacks, non-zero counting, optimization, puzzle solvers, known plaintext and trained networks, on real video, and against the published schemes under the same conditions.

A. CONTRIBUTIONS

1) *In-encoder authenticated selective encryption.* A synthesizable Verilog MJPEG encoder in which a keystream of Ascon-AEAD128, the NIST lightweight standard [7], [8], is XORed onto coefficient signs and amplitudes inside the entropy coder (Section V). We prove that the operation preserves every code length (Lemma 1) and that the cipher can never stall the encoder (Proposition 3); RTL simulation confirms zero added cycles. A per-frame tag binds the coded scan to the protection level, session, frame counter and image size.
2) *A hardware form of a keyed macroblock permutation.* The strongest level, E5, hides block positions. The fabric codes one restart segment per macroblock with its own keystream and authenticates them in raster order; the ARM core sends them in a keyed per-frame order; the receiver restores the order before the tag check (Section V-E). No frame buffer is needed, the eavesdropper's picture is identical to that of a frame-level shuffle, and the order is authenticated.
3) *A parallel-lane architecture.* Restart segments make lanes independent; one, two and four lanes produce identical bytes and reach 66, 122 and 176 Mpixel/s at 100 MHz, close to the analytical bounds of Section VI-D, and close timing after place and route on the xc7z020 together with complete PYNQ-Z2 systems (Section VIII).
4) *SVE-Bench, a common yardstick.* Six protection levels and four published format-compliant schemes [3], [4], [6], [9] are scored under the same four keyless attacks on 9 sources, together with jigsaw solvers, sign-bit recovery [5], a chosen-plaintext attack [10], trained U-Nets and a person detector (Section IX).
5) *A closed feedback loop.* An authenticated feedback channel, a forward-secure key ratchet and an adaptive policy raise protection when the receiver reports attacks; we evaluate 11 active attacks and benign impairments, and compare a threshold rule with a contextual bandit over 10 paired seeds (Section X).
6) *A verification flow that keeps hardware and claims honest.* A Python golden model is the referee for every RTL stage; Ascon passes all official known-answer tests; every number in this paper is generated from stored results by a script (Section VII).

B. ORGANIZATION

Section II reviews related work and how hardware designs in the field are evaluated. Section III gives the mathematics of baseline JPEG, Ascon and the leakage metrics. Section IV states the threat model. Section V derives the cryptographic design, Section VI the hardware architecture down to gate level, and Section VII the verification. Sections VIII to XI report hardware, security, protocol and further results; Section XII discusses limitations and concludes.

II. RELATED WORK

A. SELECTIVE ENCRYPTION OF JPEG

Van Droogenbroeck and Benedett encrypted the appended bits of selected AC coefficients of baseline JPEG, leaving the Huffman code words in place, so that the stream stays compliant at a constant bit rate [3]. Puech and Rodrigues encrypted selected DCT coefficients of medical images with AES in OFB mode [11]. Auer et al. encrypted baseline JPEG at bitstream level, permuting Huffman code-word/value pairs and toggling value bits, with an AES-OFB generator at more than 25 VGA frames per second, preserving length and format [4]. Massoudi et al. survey the field and note that visual distortion is subjective and does not measure security: replacement and chosen-plaintext attacks recover more than the scrambled image suggests [2]. Selective encryption has also been extended to H.264/AVC and HEVC [12].

B. ATTACKS ON FORMAT-COMPLIANT ENCRYPTION

Lin et al. show that DCT sign bits can be recovered by linear programming [5], which bounds what sign-only encryption can promise. Hirose et al. analyse bitstream-level JPEG encryption that shuffles blocks delimited by restart markers; the number of non-zero coefficients per block, which no field encryption hides, recovers contours unless the segments are permuted, and partly again when the restart interval is large [6]. Our strongest level uses the same mechanism with a keyed, per-frame permutation. Li et al. recover block-shuffled JPEG with a designed plaintext that reveals the permutation through its per-block non-zero counts, which then inverts any image shuffled with the same permutation [10]. Learned attackers are a threat beyond hand-made ones: Sirichotedumrong and Kiya find that a GAN-based ciphertext-only attack recovers visual content from most image-protection schemes designed for privacy-preserving deep learning [13]. Block permutations also invite jigsaw solvers, from greedy placement [14] to minimum-spanning-tree assembly with gradient compatibility [15].

C. HARDWARE JPEG ENCODERS WITH ENCRYPTION

Kishore et al. added PN-sequence block and symbol scrambling to an FPGA JPEG encoder for 256×256 grey images on Stratix II and IV; they describe the extra hardware as about 1 %, while their tables show 13–17 % more ALUTs, twice the memory bits and 9–10 % fewer frames per second with it [9]. Bağbaba and Örs combined an FPGA JPEG encoder with TEA encryption of the DC coefficients, or of the DC

and first five AC coefficients, of every block [16]. Neither authenticates the frames, and both use key material that does not change from frame to frame. Qiu et al. XOR raw video frames with two one-dimensional chaotic sequences in real time on a satellite payload (a Raspberry Pi 4B) and also implement the maps on an FPGA; their output is neither a compressed stream nor authenticated [17]. Commercial JPEG encoder cores process one sample per clock at 150–166 MHz without cryptography [18], [19].

D. LIGHTWEIGHT AUTHENTICATED ENCRYPTION IN HARDWARE

Ascon, now standardized by NIST in SP 800-232 [7], [8], is small and fast in hardware: a core with one unrolled round takes about 7 kGE, and a threshold implementation against power analysis costs about 3.1 times the area of the unprotected low-area core [20]. Khairallah and Yadhunathan compare FPGA message-authentication codes built on the Ascon permutation on Artix-7 [21]. Gladis Kurian and Chen implement Ascon on an Artix-7 with replay detection through a Bloom filter in block RAM, which has no false negatives but a small false-positive rate (0.17 % measured) [22].

E. TRAFFIC ANALYSIS OF ENCRYPTED VIDEO

Sizes and timing of encrypted streams reveal motion [23] and which video is played [24]; traffic shaping hides IoT activity at up to $18\times$ the bandwidth for a camera [25]. No encryption of content changes this, so we measure it separately (Section XII).

F. HOW HARDWARE DESIGNS IN THIS FIELD ARE EVALUATED

Table 1 summarizes what the hardware papers above report and how. Most stop at synthesis or place and route: Kishore et al. give post-synthesis resources and compute the maximum frame rate as the inverse of the coding time of one frame at the synthesized clock [9]; Bağbaba and Örs target a Virtex-6 ML605 kit and evaluate the encryption by PSNR and RMSE [16]; Khairallah and Yadhunathan report Vivado synthesis results including power [21]; Gladis Kurian and Chen implement their design on an Arty A7-100T, show replay detection on the board with LED indicators and report on-chip power split into static and dynamic parts [22]. The most complete measurement methodology we found is the Zynq-7020 testbed of Farahmand et al. for cryptographic cores: an AXI DMA streams data between DDR and the core, an AXI timer measures hardware and software time to one clock cycle, every hardware output is compared with software, and the clock is raised by binary search to the highest frequency that still gives correct results [26]. We follow the same principles: bit-exact comparison with a reference for every frame, cycle-accurate throughput, post-route timing and power with simulated switching activity, and a board protocol with the same DMA structure (Section VIII-D).

G. GAP

We found no design that (i) places a standardized lightweight AEAD inside a hardware JPEG encoder at the entropy-coding stage, (ii) keeps every protection level playable while authenticating each frame together with its metadata, (iii) hides block positions in hardware without a frame buffer, and (iv) closes the loop from the receiver back to the encoder so that the protection level follows observed attacks. We also could not find an evaluation that scores published format-compliant schemes and new ones under the same attacks, including learned attackers, on real video.

III. BACKGROUND AND NOTATION

Table 2 lists the symbols used in the rest of the paper. All arithmetic of this section is the bit-exact contract between the golden model and the RTL: the hardware computes exactly these integer formulas, and the tests compare every intermediate value.

A. BASELINE JPEG

Colour and sampling. Each pixel (R, G, B) is converted with 8-bit fixed-point coefficients and clamped to $[0, 255]$:

$$\begin{aligned} Y &= \lfloor (77R + 150G + 29B + 128)/2^8 \rfloor, \\ C_b &= \lfloor (-43R - 85G + 128B + 32896)/2^8 \rfloor, \\ C_r &= \lfloor (128R - 107G - 21B + 32896)/2^8 \rfloor, \end{aligned} \tag{1}$$

and every 2×2 group of chroma samples a, b, c, d is averaged as $\lfloor (a + b + c + d + 2)/4 \rfloor$ (4:2:0). A 16×16 macroblock (MCU) therefore yields six 8×8 blocks, four of luma and one of each chroma component, i.e. 384 samples for 256 pixels, or 1.5 samples per pixel.

Transform. The two-dimensional DCT of a block X of level-shifted samples is

$$F(u, v) = \tfrac{1}{4} c(u) c(v) \sum_{y=0}^{7} \sum_{x=0}^{7} X(y, x)\, \phi_v(x)\, \phi_u(y) \tag{2}$$

with $\phi_k(x) = \cos \frac{(2x+1)k\pi}{16}$, $c(0) = 1/\sqrt{2}$ and $c(k) = 1$ otherwise. The hardware uses the separable integer form with the 13-bit matrix $C(u, x) = \mathrm{round}\big(2^{12} c(u) \phi_u(x)\big)$ and two rounding passes,

$$\begin{aligned} G(y, v) &= \big\lfloor \big(\textstyle\sum_x C(v, x)\, (X(y, x) - 128) + 2^{10}\big)/2^{11} \big\rfloor, \\ F(u, v) &= \big\lfloor \big(\textstyle\sum_y C(u, y)\, G(y, v) + 2^{14}\big)/2^{15} \big\rfloor, \end{aligned} \tag{3}$$

whose scale is $2^{13} \cdot 2^{13}/2^{26} = 1$, so F approximates the exact DCT; the first pass keeps two fractional bits, which keeps the result within one unit of the exact transform.

Quantization. Division by the table entry $Q(u, v)$ is replaced by a 17-bit reciprocal $R(u, v) = \mathrm{round}(2^{16}/Q(u, v))$,

$$q = \mathrm{sgn}(F) \big\lfloor (|F| R + 2^{15})/2^{16} \big\rfloor, \tag{4}$$

clamped to $[-1024, 1023]$ for DC and $[-1023, 1023]$ for AC. With $R = 2^{16}/Q + \delta$, $|\delta| \le \frac{1}{2}$, the product deviates from $|F|/Q$ by $|F|\,|\delta|/2^{16} \le 2^{11}/2^{17} = 2^{-6}$, so (4) equals the

TABLE 1. How related hardware designs were evaluated (from each source).

Work	Platform	Area / timing	Throughput	Correctness, power
Kishore [9]	Stratix II/IV	synthesis	computed from clock	PSNR; no power
Bağbaba [16]	Virtex-6 ML605	synthesis	–	PSNR, RMSE; no power
Khairallah [21]	Artix-7	Vivado synthesis	computed	Vivado power
Kurian [22]	Arty A7-100T board	Vivado	latency in simulation	LEDs on board; on-chip power
Farahmand [26]	ZedBoard (Zynq-7020)	place and route	AXI timer, DMA	vs. software; max. frequency search
Ours	PYNQ-Z2 (Zynq-7020)	post-route, OOC + system	cycle-accurate RTL	bit-exact vs. model in simulation and on the board; SAIF power; board power [TODO: A4]

TABLE 2. Notation.

Symbol	Meaning
W, H	frame width and height (multiples of 16)
$M = WH/256$	number of 16×16 macroblocks (MCUs)
v, s, $\mathcal{A}(v)$	quantized coefficient, its size category, its appended field
t, σ, e	frame counter (63 bits), session identifier (64 bits), key epoch
$N(\sigma, t, b)$	128-bit nonce, $b \in \{0, 1\}$ selects keystream or tag
K_{m}, K_e, K_{fb}	master key, video key of epoch e, feedback key
K_{seg}, K_π	per-frame segment key, E5 permutation key
Z_s	keystream of restart segment s
T_t, P_t	tag of frame t, its 26-byte authenticated prefix
π_t	E5 permutation of the M segments of frame t
L, c_L, Θ_L	number of lanes, cycles per pixel, throughput
$\oplus$, $\|$, $\ggg$	XOR, concatenation, rotation to the right

rounded quotient except when $|F|/Q$ lies within 2^{-6} of a half-integer. One DSP slice per lane replaces a divider.

Entropy coding. The 64 coefficients are read in zig-zag order. The DC coefficient is coded as the difference $d = q_0 - q_0'$ to the previous block of the same component; each non-zero AC coefficient as a pair (run of preceding zeros, size category) followed by an appended field. For $v \neq 0$ the size category is

$$s(v) = \lfloor \log_2 |v| \rfloor + 1, \qquad 2^{s-1} \leq |v| \leq 2^s - 1, \tag{5}$$

($s \leq 11$ for DC differences, $s \leq 10$ for AC), and the appended field is the s-bit one's-complement code

$$\mathcal{A}(v) = \begin{cases} v, & v > 0, \\ v + 2^s - 1, & v < 0. \end{cases} \tag{6}$$

The pair goes into a Huffman code (we use the tables of Annex K of T.81 [27], [28]); a run of 16 zeros is the symbol ZRL and the rest of a block after its last non-zero coefficient is EOB. Two properties follow and our design rests on both. First, the appended field can be changed without changing the Huffman code or the stream length, as long as s is kept. Second, the Huffman symbols, and with them the run lengths and size categories of every non-zero coefficient, stay visible to anyone: they carry the outline of the picture.

Restart markers. A restart marker RST_m (0xFFD0 + m, $m = 0 \ldots 7$ cyclically) ends a restart segment: the bit stream is padded to a byte with 1-bits, all DC predictors are reset to zero, and the next segment can be decoded without any knowledge of the previous one [27]. A DRI segment in the header gives the number of MCUs per segment. Inside segments, every byte 0xFF is followed by a stuffed 0x00, so a marker cannot occur in coded data. Restart segments make both the parallel lanes (Section VI-D) and the block permutation of E5 (Section V-E) possible.

B. ASCON-AEAD128

Ascon [8], standardized by NIST in SP 800-232 [7], is built on a 320-bit permutation applied to a state $S = x_0 \,\|\, x_1 \,\|\, x_2 \,\|\, x_3 \,\|\, x_4$ of five 64-bit words. One round is $p = p_L \circ p_S \circ p_C$:

- constant addition p_C: $x_2 \leftarrow x_2 \oplus c_i$ with $c_i = 16(15 - i) + i$ for round i (12 rounds: $i = 0 \ldots 11$; 8 rounds: $i = 4 \ldots 11$);
- substitution p_S: a 5-bit S-box applied to the 64 bit slices $(x_0[j], \ldots, x_4[j])$ in parallel. In bitsliced form, with $a_0 = x_0 \oplus x_4$, $a_4 = x_4 \oplus x_3$, $a_2 = x_2 \oplus x_1$ and $(t_0, \ldots, t_4) = (a_0, x_1, a_2, x_3, a_4)$,
$$b_i = t_i \oplus (\neg t_{i+1} \wedge t_{i+2}) \quad \text{(indices mod 5)}, \tag{7}$$
followed by $x_0' = b_0 \oplus b_4$, $x_1' = b_1 \oplus b_0$, $x_2' = \neg b_2$, $x_3' = b_3 \oplus b_2$, $x_4' = b_4$;
- linear layer p_L: $x_i \leftarrow \Sigma_i(x_i) = x_i \oplus (x_i \ggg r_i) \oplus (x_i \ggg r_i')$ with $(r_i, r_i') = (19, 28), (61, 39), (1, 6), (10, 17), (7, 41)$.

Ascon-AEAD128 takes a 128-bit key K and nonce N. It initializes $S \leftarrow p^{12}(IV \,\|\, K \,\|\, N) \oplus (0^{192} \,\|\, K)$; absorbs the associated data in 128-bit blocks, each XORed into $x_0 \,\|\, x_1$ and followed by p^8; flips a domain-separation bit in x_4; encrypts each 128-bit plaintext block P_i as $C_i = P_i \oplus (x_0 \,\|\, x_1)$, sets $x_0 \,\|\, x_1 \leftarrow C_i$ and applies p^8; and finalizes with $S \leftarrow p^{12}(S \oplus (0^{128} \,\|\, K \,\|\, 0^{64}))$ and the tag $T = (x_3 \,\|\, x_4) \oplus K$. Padding appends a single 1 and zeros to the last partial block. Bytes are loaded little-endian. The standard claims 128-bit security for confidentiality and integrity in the nonce-respecting setting and requires a new key when its data or decryption-failure limits are reached [7].

We use the construction in two ways. As a keystream generator: for an all-zero message the ciphertext blocks are the rate words $x_0 \,\|\, x_1$ after each p^8, i.e. a stream of pseudo-random bits under (K, N). As a MAC: the tag of an empty message with the data to authenticate as associated data. Both need the same core; one round per clock cycle gives 128 keystream bits every nine cycles (one cycle to absorb, eight rounds).

C. LEAKAGE AND IMAGE-ENCRYPTION METRICS

Let I be the original luma and $\hat{I}$ a picture an attacker reconstructs. Edge correlation, our main measure of outline leakage, is the Pearson correlation between Sobel gradient magnitudes,

$$\rho_E = \frac{\mathrm{cov}\big(|\nabla I|, |\nabla \hat{I}|\big)}{\sigma_{|\nabla I|}\, \sigma_{|\nabla \hat{I}|}}, \tag{8}$$

computed over all pixels; $\rho_E = 0$ means no outline, and it is insensitive to brightness and contrast, which a learned attacker recovers from global statistics. We also report SSIM [29]. For comparability with image-encryption papers we compute, between two pictures C_1, C_2 of N pixels, $\mathrm{NPCR} = \frac{100}{N}\sum_i [C_1(i) \neq C_2(i)]$ and $\mathrm{UACI} = \frac{100}{N}\sum_i |C_1(i) - C_2(i)|/255$, the histogram entropy $-\sum_k p_k \log_2 p_k$ and the correlation of adjacent pixels. These metrics were designed for ciphers that turn the whole image into noise; a format-compliant scheme keeps the stream decodable by design, so we read them together with (8).

IV. THREAT MODEL AND SECURITY GOALS

System. A camera node (the PYNQ-Z2 board) encodes video and sends one JPEG per frame over UDP to a viewer; the viewer returns heartbeats and alerts over a feedback channel. Both ends hold a 128-bit master key K_m provisioned at installation. As in SRTP, whose key management is a separate protocol [1], and in every scheme we compare with [3], [4], [6], [9], [16], key distribution is out of scope; a standard key exchange on the ARM core at session start is the natural extension (Section XII).

Adversary. The attacker sits on the network path. It observes all traffic and may modify, drop, delay, replay, reorder or inject both video and feedback datagrams; it does not hold the keys. It knows the design, the code and the format, may know some frames in clear (known plaintext), may choose content that the camera films (chosen plaintext), and may train neural networks on any amount of public video. A passive eavesdropper sends nothing, so it cannot be detected; against it only the configured minimum protection level helps. Physical and side-channel attacks on the device are out of scope; a deployment that needs them can use a threshold implementation of Ascon, which Groß et al. built at about 3.1 times the area of their unprotected low-area core [20].

Assets and goals.

- G1 Content confidentiality: a keyless observer should not recover the shape of the scene. Format compliance means that the frame size, the image dimensions, the protection level and coarse statistics are public by design; we measure what else leaks (Section IX) and what the public parts reveal (Section XII).
- G2 Integrity and authenticity of every frame together with its metadata (level, session, counter, size): any change must be rejected.
- G3 Freshness: no replayed or re-ordered old frame is shown as current.
- G4 Availability signal: the viewer must learn that the camera is alive, and the camera that the viewer receives; silence must be treated as an attack.
- G5 Forward secrecy across epochs: a video key captured later does not decrypt earlier epochs.

G2–G5 are enforced cryptographically (Section V); G1 is a property of the protection level, which we measure empirically against the strongest attackers we could build. The receiver accepts a frame only if its tag verifies, so a wrong key, a modified frame or a forged frame is rejected rather than shown as noise; a viewer without the key can still play the stream, which is the point of format compliance.

V. CRYPTOGRAPHIC DESIGN

A. OVERVIEW

Fig. 1 shows the system. The ARM core of the Zynq places a frame in DDR, writes the per-frame registers (level, counter, size) and starts the DMA; the fabric reads the pixels, codes them into a JPEG scan in which the selected coefficient fields are encrypted, computes the frame tag over the metadata and the scan, and writes the scan back. The ARM core reads the tag, wraps the scan in standard JPEG headers and an application segment (APP11) that carries the metadata and the tag, reorders the segments if the level is E5, and sends the frame in UDP datagrams. All per-byte cryptographic work happens in the fabric; the ARM touches each output byte once.

B. KEYS, NONCES AND THEIR UNIQUENESS

All keys derive from K_m with one primitive, the Ascon tag used as a pseudorandom function (Fig. 2):

$$\mathrm{KDF}(K, \ell, c) = \mathrm{MAC}_K\big(N(0, c, 1), \ell\big), \tag{9}$$

the tag of an empty message with label ℓ as associated data, under a nonce with session identifier 0, which no frame carries. The feedback key is $K_\mathrm{fb} = \mathrm{KDF}(K_\mathrm{m}, \text{feedback}, 0)$; the video key of epoch e follows the ratchet

$$K_0 = \mathrm{KDF}(K_\mathrm{m}, \text{video}, 0), \qquad K_{e+1} = \mathrm{KDF}(K_e, \text{ratchet}, e{+}1), \tag{10}$$

and each end erases K_e once it has moved on, so a key captured in epoch e does not decrypt epochs before e (goal G5). Rekeying needs no handshake: the epoch is part of the authenticated session identifier of every frame, so the receiver ratchets forward when a frame from a later epoch verifies. Reusing the verified Ascon core for key derivation avoids a second primitive in hardware; the extendable-output function of SP 800-232 is the textbook alternative.

The nonce is the 128-bit string

$$N(\sigma, t, b) = \underbrace{\sigma}_{x_3,\ 64\ \text{bits}} \,\|\, \underbrace{t \,\|\, b}_{x_4,\ 63{+}1\ \text{bits}}, \qquad \sigma = \beta \,\|\, e, \tag{11}$$

with β a 48-bit random base drawn at every boot ($\beta \neq 0$), e the 16-bit epoch, t the frame counter and b the purpose bit (0 for keystream derivation, 1 for the tag). The words map directly onto the Ascon state words x_3 and x_4.

Proposition 1 (Nonce uniqueness)**.** *Within one boot, no key is used twice with the same nonce, provided the sender never writes the same frame counter twice in an epoch.*

Proof: Under a video key K_e the nonces used are $N(\sigma, t, 0)$ for the segment key, $N(\sigma, t, 1)$ for the tag, $N(0, 0, 0)$ for the permutation key and $N(0, c, 1)$ for derivations; $\sigma \neq 0$ because $\beta \neq 0$, b separates the first two, and t is fresh per frame. Under a segment key K_seg, which is a

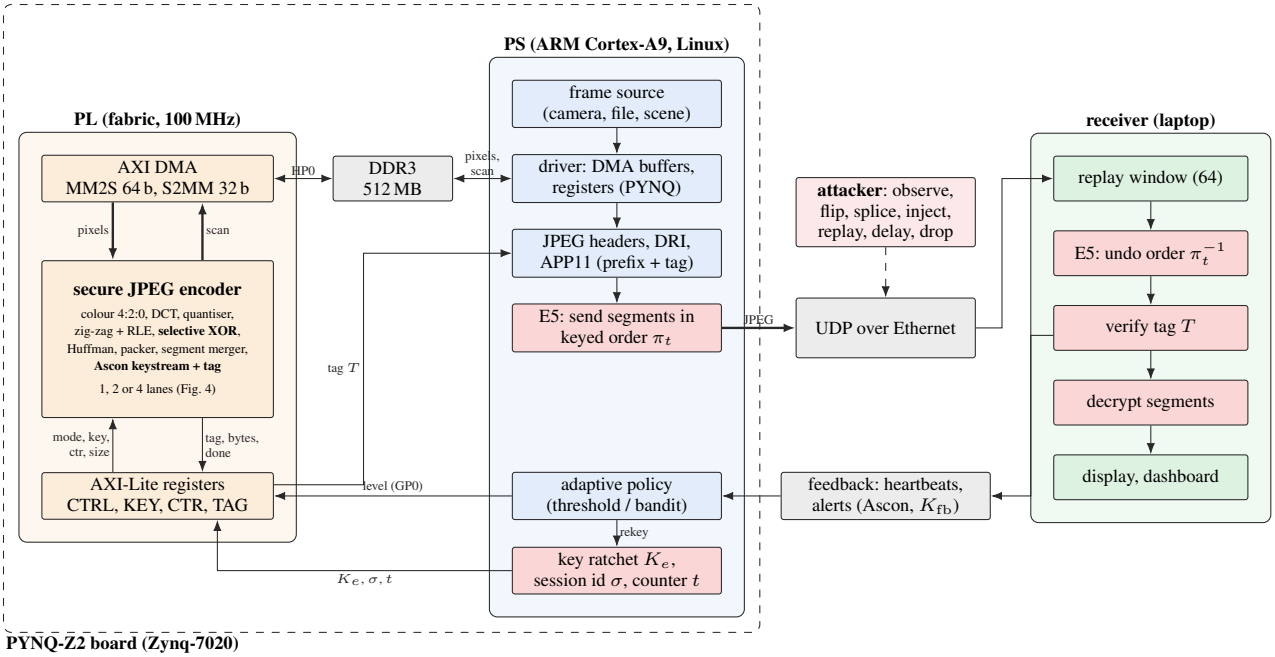
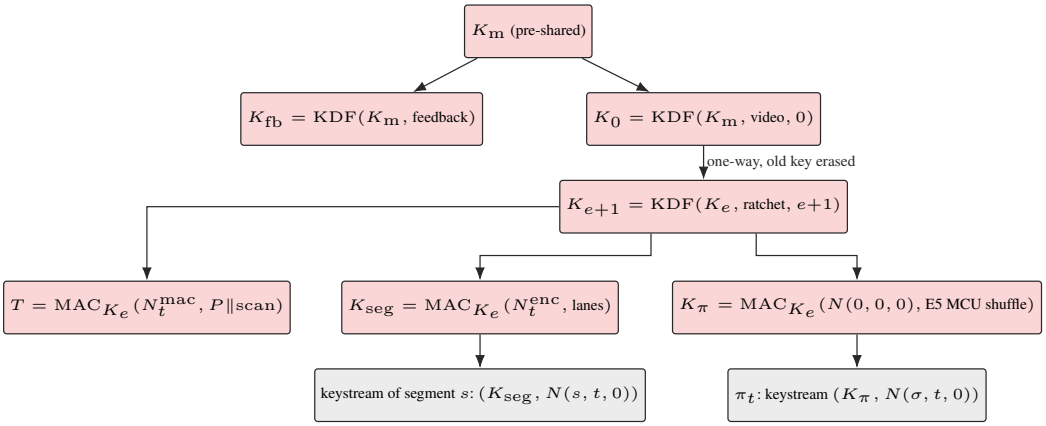


FIGURE 1. System architecture. The programmable logic (PL) encodes, encrypts and tags every frame; the ARM core (PS) moves pixels and scan through DDR with an AXI DMA, adds the JPEG headers and the APP11 segment with the tag, sends E5 segments in the keyed order, and runs the key ratchet and the adaptive policy. The receiver undoes the order, verifies the tag, decrypts, and reports back over an authenticated feedback channel. Shaded red: cryptographic operations.



Nonce $N(\sigma, t, b)$, 128 bits = Ascon state words $x_3 \| x_4$: $x_3 = \sigma$ = 48-bit random base (new at every boot) $\|$ 16-bit key epoch e; $x_4 = t \| b$, t = 63-bit frame counter, b = purpose (0 keystream, 1 tag). $N_t^{\mathrm{enc}} = N(\sigma, t, 0)$, $N_t^{\mathrm{mac}} = N(\sigma, t, 1)$, P = 26-byte prefix (version, mode, σ, t, W, H); the KDF uses $\sigma = 0$, which no frame carries.

FIGURE 2. Key and nonce hierarchy. One pre-shared master key yields the feedback key and the epoch-0 video key; a one-way ratchet yields every later epoch. Per frame, the video key derives the segment key (keystream) and the E5 permutation key and authenticates the frame.

pseudorandom function of (K_e, σ, t) and therefore distinct for every frame, the nonces $N(s, t, 0)$ differ in the segment index s. Under K_π the nonces $N(\sigma, t, 0)$ differ in t. A rekey changes both K_e and e, hence σ. ■

Across boots, two runs that start from the same epoch use the same key and are separated only by β. For B boots under one master key the probability that any two share β is at most

$$p_{\mathrm{coll}} \le \binom{B}{2} 2^{-48} \approx 1.8 \times 10^{-7} \quad \text{for } B = 10^4, \qquad (12)$$

about one boot a day for 27 years. A deployment that cannot accept this persists the counter or the epoch across reboots instead. The 63-bit counter cannot wrap in practice (2^{63} frames at 30 frames/s last about 10^{10} years).

C. SELECTIVE ENCRYPTION INSIDE THE ENTROPY CODER

Every level carries the per-frame tag. Table 3 lists which fields each level encrypts. E0 encrypts nothing (authentication only), E1 the AC sign bits, E2 also the DC-difference signs (the default minimum), E3 also the AC amplitude bits, E4 also the DC amplitude bits, and E5 encrypts the fields of

TABLE 3. Protection levels: encrypted fields of every non-zero coefficient (sign: the whole field is inverted; amplitude: its $s-1$ low bits are XORed) and block positions.

Level	AC sign	DC sign	AC ampl.	DC ampl.	Positions
E0					
E1	✓				
E2	✓	✓			
E3	✓	✓	✓		
E4	✓	✓	✓	✓	
E5	✓	✓	✓	✓	keyed per frame

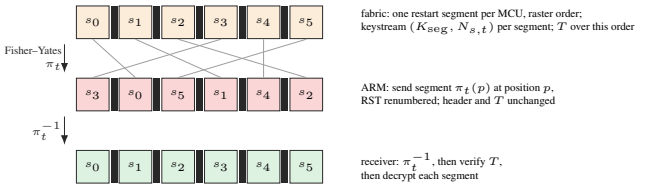


FIGURE 3. E5 in hardware form. The fabric codes one restart segment per MCU in raster order, each with its own keystream, and tags them in that order; the ARM sends segment $\pi_t(p)$ at wire position p; the receiver restores raster order before checking the tag. Black bars: restart markers.

E4 and permutes the macroblocks (Section V-E).

For a selected symbol with size category s and keystream bits $z_0 z_1 z_2 \ldots$ (taken in order from the segment keystream), the encryption stage computes

$$\mathcal{A}' = \mathcal{A} \oplus \underbrace{\gamma z_0 (2^s - 1)}_{\text{sign}} \oplus \underbrace{\alpha \sum_{k=1}^{s-1} z_k \, 2^{k-1}}_{\text{amplitude}}, \tag{13}$$

where $\gamma, \alpha \in \{0, 1\}$ say whether the level selects the sign and the amplitude of this coefficient (DC or AC). It consumes

$$n(s) = \gamma + \alpha(s-1) \le 11 \tag{14}$$

keystream bits; decryption is the same XOR.

Lemma 1 (Format preservation)**.** *For every $v \neq 0$ and every keystream,* (13) *maps $\mathcal{A}(v)$ to $\mathcal{A}(v')$ for some v' with $s(v') = s(v)$. Hence the Huffman symbol (run, s), the code length and the length of the scan are unchanged, and the encrypted scan is a valid baseline JPEG scan.*

Proof: Let $m = 2^s - 1$. For $v > 0$, $\mathcal{A}(v) = v < 2^s$, so $\mathcal{A}(v) \oplus m = m - v = \mathcal{A}(-v)$ by (6); the case $v < 0$ is symmetric. The sign term therefore maps $\mathcal{A}(v)$ to $\mathcal{A}(-v)$, and $|-v| = |v|$. The amplitude term changes only bits $0 \ldots s-2$ of the field, never bit $s-1$. By (6) and (5), bit $s-1$ of $\mathcal{A}(v)$ is 1 exactly when $v > 0$, and the low $s-1$ bits encode $|v| - 2^{s-1}$ (positive v) or its complement (negative v); any value of them gives a magnitude in $[2^{s-1}, 2^s - 1]$, i.e. size category s, with the same sign. Neither term touches the run or the Huffman code. ∎

Two consequences matter. The scan has exactly the length of the unencrypted one, so encryption needs no buffer and adds no bandwidth; and the sign term is an exact negation, not a bit flip of the field's top bit, which would change the magnitude (an earlier version of our model did that, see Section VII). The lemma also shows what cannot be hidden: the run lengths and size categories, i.e. which coefficients are non-zero and roughly how large, stay public at every level E0–E4.

D. SEGMENT KEYSTREAMS AND THE FRAME TAG

Every frame is coded as restart segments (one per MCU row, or one per MCU in E5). Each segment s of frame t has its own keystream under a per-frame segment key:

$$K_{\text{seg}} = \text{MAC}_{K_e}\big(N(\sigma, t, 0), \text{“lanes”}\big), \tag{15}$$

$$Z_s = \mathcal{E}_{K_{\text{seg}}}\big(N(s, t, 0), 0^{\ell}\big), \tag{16}$$

taken least significant bit first. Seeking to a segment therefore costs one initialization and nothing else, which is what lets lanes encrypt independently (Section VI-D). The tag authenticates a 26-byte prefix and every byte of the scan, including restart markers:

$$T_t = \text{MAC}_{K_e}\big(N(\sigma, t, 1),\ P_t \,\|\, \text{scan}_t\big), \tag{17}$$

$$P_t = \texttt{SJV1} \,\|\, \text{version} \,\|\, \text{level} \,\|\, \sigma \,\|\, t \,\|\, W \,\|\, H.$$

The ARM core places P_t (without W, H, which are in the frame header) and T_t in a 38-byte APP11 segment that ordinary viewers skip. Binding the level into the tag matters: an earlier version authenticated only the scan, and a frame could be downgraded from E2 to E1 undetected, so the receiver showed a wrongly decrypted frame as authentic. The receiver accepts a frame only if (17) verifies and the counter passes the replay window (Algorithm 2); a single flipped bit anywhere in the prefix or scan is rejected except with probability 2^{-128} per forgery attempt.

E. E5: A KEYED PERMUTATION OF MACROBLOCKS IN HARDWARE FORM

Lemma 1 shows that field encryption cannot hide where the non-zero coefficients are; the number of non-zero coefficients per block alone rebuilds the contours (the non-zero counting attack, NZCA [6]). E5 therefore also hides where each macroblock belongs. Permuting macroblocks before entropy coding would need a frame buffer in the fabric. Restart segments make a cheaper split possible (Fig. 3):

1) the fabric codes every MCU $s = r \cdot (W/16) + c$ as its own restart segment, encrypted like E4 with keystream Z_s of (16), and authenticates the segments in raster order with (17) (level byte 5);
2) the ARM splits the scan at the restart markers and sends segment $\pi_t(p)$ at position p, renumbering the markers $\text{RST}_{p \bmod 8}$ in output order; headers, APP11 and tag are unchanged, and DRI = 1;
3) the receiver splits the received scan, restores raster order with π_t^{-1}, verifies T_t and only then decrypts.

Algorithm 1 E5 order π_t (ARM core and receiver)

Require: K_π, σ, t, number of segments M

$Z \leftarrow$ keystream of $(K_\pi, N(\sigma, t, 0))$; $\pi \leftarrow (0, 1, \ldots, M-1)$
for $i = M-1$ **down to** 1 **do**
  $b \leftarrow \lfloor \log_2 i \rfloor + 1$ ▷ bits to express i
  repeat $j \leftarrow$ next b bits of Z
  until $j \le i$
  swap $\pi(i)$ and $\pi(j)$
end for
return π ▷ wire position p carries segment $\pi(p)$

The permutation is drawn from its own key, $K_\pi = \mathrm{MAC}_{K_e}(N(0,0,0), \text{“E5 MCU shuffle”})$, with the Fisher–Yates shuffle (Algorithm 1) on the keystream of $(K_\pi, N(\sigma, t, 0))$; uniform indices come from rejection sampling.

Proposition 2 (Uniform per-frame permutation)**.** *If the keystream bits are uniform and independent, Algorithm 1 returns each of the $M!$ permutations with probability $1/M!$, and it consumes on average*

$$\bar{n}_\pi = \sum_{i=1}^{M-1} b_i \frac{2^{b_i}}{i+1} < 2 \sum_{i=1}^{M-1} b_i \tag{18}$$

keystream bits, $b_i = \lfloor \log_2 i \rfloor + 1$.

Proof: A b_i-bit draw is uniform on $\{0, \ldots, 2^{b_i} - 1\}$; conditioned on $j \le i$ it is uniform on $\{0, \ldots, i\}$, and the number of draws is geometric with success probability $(i+1)/2^{b_i} > 1/2$. Fisher–Yates with uniform $j \in \{0, \ldots, i\}$ at every step produces every permutation exactly once among the $\prod_{i=1}^{M-1}(i+1) = M!$ equally likely choice sequences. ■

At 320×240 ($M = 300$) the permutation carries $\log_2 M! = 2{,}041$ bits of entropy and costs on average 3,198 keystream bits (1.57 times the entropy); at 640×480 ($M = 1{,}200$) $\log_2 M! = 10{,}550$ bits and 16,322 keystream bits. The permutation changes with every frame because t enters the nonce, so knowing the order of one frame tells nothing about the next; schemes with a fixed permutation fail exactly here (Section IX-C).

Authentication of the order. Because the tag covers the raster-order scan and the receiver computes π_t^{-1} from the authenticated (σ, t), moving, swapping, dropping or duplicating a segment on the wire changes the associated data of (17) and the frame is rejected. Sending the frame unshuffled is also rejected, since the receiver always applies π_t^{-1}.

Equivalence. An eavesdropper sees the same multiset of encrypted macroblocks at uniformly random positions as with a frame-level shuffle; the only difference is that each segment carries its own keystream instead of a slice of one frame keystream, which is invisible without the key. In our measurements the eavesdropper's pictures of both forms are pixel-identical (Section IX-C).

Algorithm 2 Replay window (64 entries, after RFC 3711)

Require: counter t; state: highest counter τ, bitmap w

if $t > \tau$ **then**
  $w \leftarrow ((w \ll (t - \tau)) \vee 1) \bmod 2^{64}$; $\tau \leftarrow t$; **return** accept
end if
$o \leftarrow \tau - t$
if $o \ge 64$ **or** bit o of w is set **then return** reject
end if
set bit o of w; **return** accept

Cost. Each segment adds a two-byte marker, on average half a byte of padding, and loses DC prediction across macroblocks, so the overhead over E4 is roughly

$$\Delta B \approx M\left(2 + \tfrac{1}{2}\right) + \Delta_{\mathrm{DC}} \text{ bytes per frame}, \tag{19}$$

where Δ_{DC} is the extra length of DC differences coded from zero. Measured, E5 costs 8–23 % over E4 at 320×240 (Section IX-H); the fabric needs no frame buffer and no extra cipher core, and the ARM touches each output byte once.

F. AUTHENTICATED FEEDBACK AND THE REPLAY WINDOW

The receiver returns a heartbeat every 0.5 s and rate-limited alerts (tag failures, replays, malformed or stale frames, loss statistics). Feedback messages are Ascon-AEAD128 encrypted under K_{fb} with their own session and counter, so the attacker can neither read nor forge them, and they pass their own replay window. Both directions use the sliding window of SRTP [1] with 64 entries (Algorithm 2). A frame older than the window, or seen before, is discarded; frames that arrive late but inside the window are accepted, which tolerates reordering on UDP. Frames older than the last frame shown are discarded as late without counting as replays, so that benign reordering does not raise alarms.

Dead-man rule. If no valid heartbeat arrives for 3 s, the sender assumes that the feedback path is attacked: it switches to the highest level and rekeys (goal G4). Rekeys are rate-limited to one per 10 s.

G. ADAPTIVE PROTECTION POLICIES

The sender's ARM core collects evidence in one-second windows: tag failures f_{int}, replays f_{rep}, forged or replayed feedback f_{fb}, junk floods f_{fl}, the dead-man state f_{dm} and a loss level $f_{\mathrm{loss}} \in \{0, 1, 2\}$ (frame loss below 5 %, between 5 and 90 %, above 90 %) judged over three windows. It condenses them into a threat score

$$\theta = \min\left(1,\ f_{\mathrm{int}} + f_{\mathrm{rep}} + 0.8 f_{\mathrm{fb}} + 0.6 f_{\mathrm{fl}} + f_{\mathrm{dm}} + 0.25 f_{\mathrm{loss}}\right), \tag{20}$$

shown on the dashboard and used by the bandit. Loss level 1 is reported as a degraded network, because more encryption cannot help against loss; only near-total suppression (level 2) counts as jamming. Two policies map evidence to a level, and neither goes below the configured minimum.

Threshold rule. Any tag failure, replay, feedback attack or flood in a window raises the highest level and holds it for 8 s after the evidence ends; suppression (loss level 2) does the same for 4 s. Both policies share a safety layer: the dead-man rule, and a rekey when the tag failures of one window reach a configured burst size.

Contextual bandit. The context c is a small discrete vector of the evidence flags and whether an attack was seen in the last 30 s; the action is a level a. After each window the bandit observes the loss

$$\ell(a) = \theta' \lambda(a) + w_c\, \kappa(a) + w_s [a \neq a_{\text{prev}}], \qquad (21)$$

where θ' is the threat score of the next window, $\lambda(a)$ the measured edge correlation of level a, $\kappa(a)$ its fraction of encrypted bits, $w_c = 0.5$ and $w_s = 0.02$. It updates $Q(c,a) \leftarrow Q(c,a) + \eta\big(-\ell(a) - Q(c,a)\big)$ with $\eta = \max(0.05, 1/N(c,a))$ (sample average, then a floor), chooses an untried action first (optimistic initialization), and otherwise explores with probability $\varepsilon_c = 0.1/\sqrt{1+n_c}$ and exploits $\arg\max_a Q(c,a)$ [30]. Its table persists across episodes, so what it learned about an attack is used the next time the same context appears.

VI. HARDWARE ARCHITECTURE

A. DATAPATH

Fig. 4 shows the encoder. All stages are joined by valid/ready handshakes: data moves on a rising edge where both are high, a sender holds its data while valid is high and ready is low, and valid never depends on ready. Every stage therefore tolerates back-pressure from the DMA without losing or corrupting data, which the testbenches check with random stalls on every interface.

Colour conversion and line buffers. `rgb2ycbcr_2px` takes two pixels per clock from the 64-bit stream, applies (1) with three multiply-add trees per pixel (the luma line buffer is split into even and odd banks so that both pixels are written in one clock), averages chroma and writes an MCU row (16 lines) into block-RAM line buffers sized for 2048-pixel lines (1024 with four lanes). It reads the samples back block by block in the order Y0 Y1 Y2 Y3 Cb Cr, one sample per clock. Its buffers hold all block RAM of the encoder.

Transform and quantizer. `dct8x8` computes (3) as two passes of an eight-tap multiply-accumulate with a transpose buffer between them, one sample per clock; the row sums are pipelined over five stages. `quantiser` implements (4) with one DSP multiplication by the reciprocal table, which the ARM writes through the register map once per session.

Zig-zag, run-length and DC coding. `zigzag_rle` stores the coefficients of a block in zig-zag order, takes the DC difference with one predictor per component, and emits symbols (is_dc, run, s, $\mathcal{A}$, EOB, component, frame-last), at most one per clock. Its predictors reset at every restart segment.

Encryption. `selective_enc` implements (13) (Section VI-B). `huffman_enc` maps each symbol to a right-aligned chunk of up to 27 bits (code followed by the field) from the Annex K tables, and `bit_packer` assembles bytes, inserts the 0x00 after every 0xFF, and at the end of a segment pads with 1-bits and marks the last byte.

Merger, tag and output. `segment_merger` copies segment s byte by byte from its lane's FIFO until the byte marked last, then writes 0xFF, 0xD0 $+(s \bmod 8)$ unless s is the last segment, and moves to the next lane. `tag_unit` absorbs the prefix and every merged byte into an Ascon core; `scan_word_packer` packs bytes into 32-bit little-endian words with TKEEP and TLAST and counts them for the OUT_BYTES register.

B. CIRCUIT OF THE ENCRYPTION STAGE

Fig. 5 is one bit slice of the stage at gate level, generated from its RTL; the handshake around it is described by the equations below. The control part decodes $\gamma = [s \neq 0] \wedge (\text{is_dc}\ ?\ \text{mode} \geq 2 : \text{mode} \geq 1)$ and $\alpha = [s \neq 0] \wedge (\text{is_dc}\ ?\ \text{mode} \geq 4 : \text{mode} \geq 3)$; mode 5 selects the same fields as mode 4 because all comparisons are $\geq$. The data part is eleven identical bit slices,

$$\mathcal{A}'_k = \mathcal{A}_k \oplus (\gamma z_0 \wedge m_k) \oplus (\alpha z_{k+1} \wedge m_{k+1}), \quad m = 2^s - 1, \qquad (22)$$

one XOR gate deep after the masks, so the stage closes timing with room to spare. The handshake accepts a symbol when $i_{\text{ready}} = (\neg o_{\text{valid}} \vee o_{\text{ready}}) \wedge (k_{\text{avail}} \geq n)$ and signals the keystream generator to advance by n bits.

Keystream window. `keystream_gen` holds two 128-bit slots, the block being consumed (u) and the next one (u'), and a pointer $p \in [0, 127]$ to the next unused bit. It presents the 16-bit window $(u' \,\|\, u) \gg p$ and its fill level $k_{\text{avail}} = \min(16, 128 - p + 128\,[u' \text{ valid}])$; taking n bits sets $p \leftarrow p+n$, and when p passes 128 the next slot moves in. A new block is requested whenever fewer than two are held or in flight, so the core runs ahead of the consumer.

Proposition 3 (The cipher never stalls the encoder)**.** *Once the first keystream block of a segment is available, the encryption stage never waits for keystream bits.*

Proof: The stage accepts at most one symbol per clock and takes $n \leq 11$ bits per symbol (14), so the demand never exceeds 11 bits per cycle. The core produces 128 bits per data block in nine cycles (one cycle to absorb, eight rounds of p^8), i.e. $128/9 > 14$ bits per cycle, and it keeps up to 256 bits buffered, more than the 16 that the window exposes. Supply exceeds the worst-case demand, so the window, once filled, can never drop below the 11 bits a symbol needs. ■

The first block costs an initialization (p^{12}, 12 cycles) and one data block (nine cycles). At the start of a frame this overlaps with filling the first MCU row (16 lines), so it is invisible; RTL simulation confirms that the cycle counts of E0–E4 differ by at most 2 cycles per frame. In the lane encoder every segment restarts its keystream, and in E5 every MCU does; a segment of one MCU takes at least 384 cycles, so a 21-cycle restart overlapped with the previous segment costs a few per cent of throughput, which we measure in Section VIII.

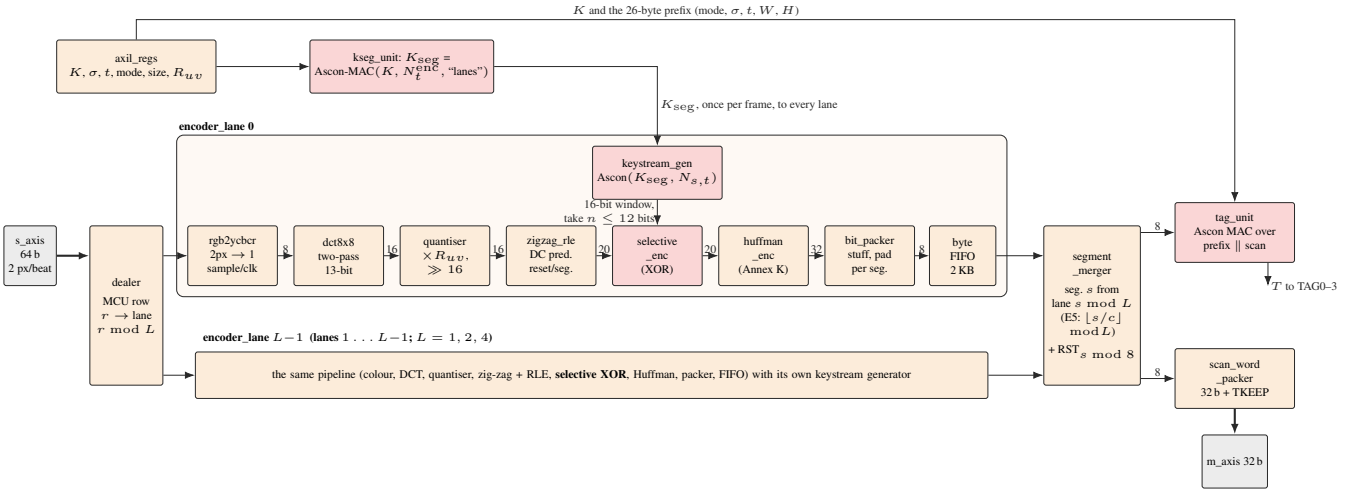


FIGURE 4. Fabric datapath of the lane encoder `secure_jpeg_lanes_top` (module names are the RTL files; numbers on arrows are bus widths in bits). The dealer sends MCU row r to lane $r \bmod L$; each lane converts two pixels per clock into one sample per clock, transforms, quantizes and entropy-codes its restart segments with its own keystream; the merger restores raster order and inserts restart markers; the tag unit authenticates the 26-byte prefix and the merged scan; the word packer writes 32-bit words to the DMA. The single-pipeline encoder `secure_jpeg_top` is one lane without dealer and merger, with one keystream per frame.

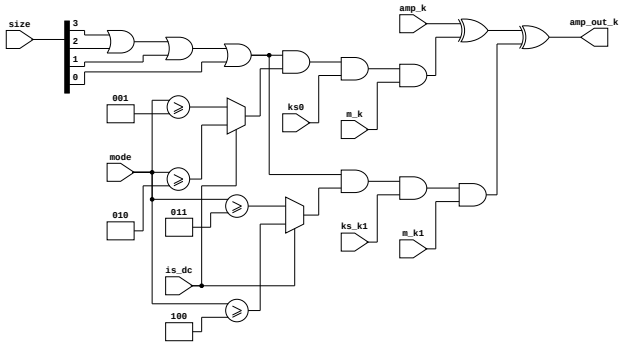


FIGURE 5. Gate-level circuit of bit slice k of `selective_enc.v`, generated from the RTL by Yosys and netlistsvg. The OR of the size bits gives $[s \neq 0]$; comparators on the mode and two multiplexers selected by is_dc form the sign enable γ (upper path) and the amplitude enable α (lower path); the two XORs implement (22). $m_k = [k < s]$ and m_{k+1} are bits of the mask $2^s - 1$; ks0 and ks_k1 are bits 0 and $k+1$ of the keystream window. A test simulates the slice exhaustively and compares it with the golden model.

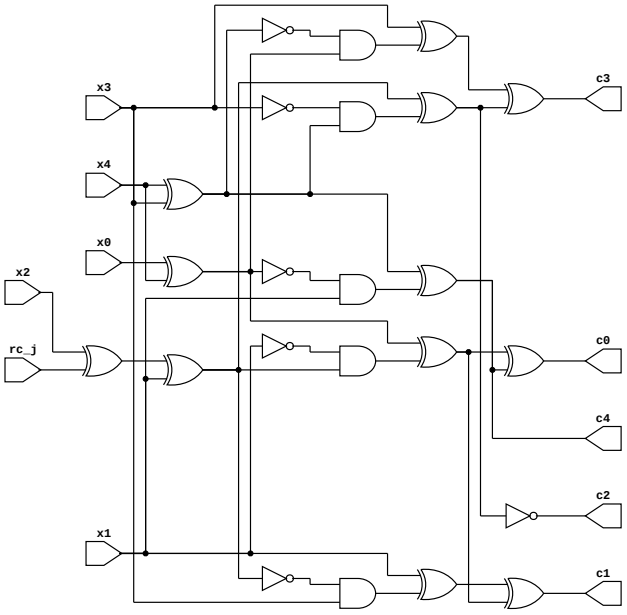


FIGURE 6. One of the 64 bit slices of the Ascon round of `ascon_round.v` (p_C and p_S), generated from the RTL by Yosys and netlistsvg: twelve XOR (one for the round-constant bit rc_j, zero for $j \geq 8$), five AND and six NOT gates. The linear layer p_L acts on whole 64-bit words (two rotations and two XORs per bit). A test checks that 64 such slices followed by p_L reproduce a round of the golden model.

C. ASCON CORE

`ascon_aead128_core` computes one round per clock cycle: a 320-bit state register, a 4-bit round counter r and the combinational round of Fig. 6, with the round constant $c = \bar{r} \,\|\, r$ (eight bits, $\bar{r}$ the complement of r), which equals $16(15-i)+i$ for round i. The p^{12} permutation counts r from 0 to 11, the p^8 permutation from 4 to 11. Counting the operators of `ascon_round.v`, one bit slice of p_S has 22 gates (eleven XOR, five AND, six NOT), so p_S is $64 \times 22 = 1408$ gates, p_L adds 2×320 two-input XORs and p_C eight. A block interface accepts associated data and message blocks with byte masks and the padding rule of the standard; the same core serves as keystream generator (message blocks of zeros, never finalized), as MAC (associated data only, empty message) and as key-derivation unit. The design instantiates one core per lane for keystreams, one for the segment key and one for the tag.

Tag unit. `tag_unit` latches the 26-byte prefix at frame start, absorbs it, then gathers scan bytes into 128-bit blocks. A block of 16 bytes arrives in at least 16 cycles at one byte per clock, while absorbing it takes nine, so the tag unit never slows the merger; the final partial block, the empty message and the p^{12} finalization add a fixed number of cycles after the last byte.

TABLE 4. AXI4-Lite register map (32-bit; `axil_regs.v`).

Offset	Name	Function
0x00	CTRL	bit 0 START (self-clearing), bits 3:1 level E0–E5
0x04	STATUS	DONE, BUSY, TAG_VALID
0x08	VERSION	0x5EC0_0200 (single pipeline); 0x5EC0_03LL (L lanes, bit 7 = E5)
0x10, 0x14	WIDTH, HEIGHT	pixels (multiples of 16)
0x18, 0x50	FRAME_CTR (lo, hi)	63-bit frame counter t
0x1C, 0x4C	SESSION_ID (lo, hi)	64-bit session identifier σ
0x20–0x2C	KEY0–3	video key K_e (write-only)
0x30	OUT_BYTES	scan bytes of the last frame
0x34–0x40	TAG0–3	tag T_t
0x44, 0x48	QT_ADDR, QT_DATA	17-bit reciprocals $R(u, v)$

D. PARALLEL LANES

One pipeline consumes one sample per clock; with 1.5 samples per pixel it needs at least 1.5 cycles per pixel. Restart segments are independent (DC predictors reset, byte alignment, own keystream (16)), so L copies of the pipeline can code segments simultaneously: the dealer sends MCU row r to lane $r \bmod L$ and the merger reads segment s from lane $s \bmod L$ (in E5, segment s belongs to MCU row $\lfloor s/c \rfloor$ with $c = W/16$ MCUs per row, so it comes from lane $\lfloor s/c \rfloor \bmod L$, tracked with row and column counters instead of a divider). The 64-bit input bus brings two pixels per clock. The cost per pixel is therefore bounded below by

$$c_L \geq \max\Big(\frac{1.5}{L}, \frac{1}{2}\Big), \qquad \Theta_L = \frac{f_{\mathrm{clk}}}{c_L}, \tag{23}$$

i.e. 1.5, 0.75 and 0.5 cycles per pixel, or at most 67, 133 and 200 Mpixel/s at 100 MHz for $L = 1, 2, 4$; beyond four lanes the input bus, not the encoder, is the limit. Two costs come on top. Before the first MCU row is complete no block can be transformed, which for a single pipeline gives the measured law

$$c_1(r) = 1.5 + \frac{A}{r}, \qquad A = 1.22, \tag{24}$$

for a frame of r MCU rows; and each segment adds a keystream restart and a byte flush. The bytes do not depend on L, because segments are merged in raster order and each one depends only on its own pixels and keystream.

E. REGISTER MAP AND DRIVER

Table 4 gives the register map. Once per session the driver writes the key, the session identifier and the 128 reciprocals; per frame it writes the size and the counter, sets CTRL to $1 + 2 \cdot \mathrm{level}$, starts the DMA receive and send channels, waits for DONE, and reads OUT_BYTES and the tag. The VERSION register tells the driver which contract the bitstream implements: bit 7 says that the fabric codes E5 segments, and only then does the driver allow level E5.

F. PYNQ-Z2 SYSTEM

Fig. 7 shows the system around the encoder. A script builds it in Vivado from the RTL (the encoder enters as a module reference, without packaging), runs synthesis, place and route and bitstream generation, and exports the hardware handoff file that the PYNQ framework reads to find the IP blocks by name. The DMA runs in simple (register) mode; the driver allocates physically contiguous buffers, copies a frame into the input buffer, and starts both channels. Everything in the system runs on one clock, which avoids clock-domain crossings.

G. TIMING CLOSURE

Three lessons from the Vivado runs are worth recording. The first implementation met timing in simulation but reached only about 47 MHz after routing, because the DCT summed a full row in one clock; splitting the sums over five pipeline stages with a stall-able handshake restored 100 MHz without changing an output bit. The second appeared only in the complete Zynq system: the DMA's output register fed the colour conversion, clamp and line-buffer write in one clock. Registering the pixel, the multiply-add sums and the clamped result moved the writes three clocks later in the same order. The third came with the lanes: the zig-zag stage, which formed the zig-zag address, read the block buffer, took the DC difference and derived the size category and field in one clock, had been the critical path of the single pipeline with tens of picoseconds of slack, and missed 100 MHz in the two-lane system. Storing coefficients in zig-zag order and splitting the coding into two handshaked stages closed timing for every configuration and raised the encoder's slack to +0.554 ns; at 125 MHz the colour converter is now the limit (WNS -0.875 ns). In all three cases the golden model was the referee: a fix was accepted only when every RTL test still matched it bit for bit.

VII. VERIFICATION METHODOLOGY

One referee. A Python golden model implements Ascon, the integer transforms of Section III, entropy coding, the encryption of (13), the container, E5 and the receiver. Two views of it exist: a frame-level one used by the network simulation and the receiver, and a stage-level one whose functions match the RTL module boundaries; a test proves the two byte-identical on random frames for every level and both contracts (single pipeline and lanes). The Ascon model passes all 1089 known-answer tests of the official file for Ascon-AEAD128, and so does the RTL core. When the model and an RTL block disagree, the model is changed only if it contradicts T.81 or SP 800-232.

Block tests. Every RTL module has a cocotb testbench that drives random inputs with random idle cycles and random back-pressure on every handshake and compares every output with the corresponding model function, reporting the first mismatch with its index, input, obtained and expected value. Throughput tests check one item per clock at full rate; mutation checks confirm that a stage which ignores back-pressure fails them.

System tests. The complete encoders are compared byte for byte with the model for all levels, with 64-bit session identifiers and counters above 2^{32}, for frames with fewer

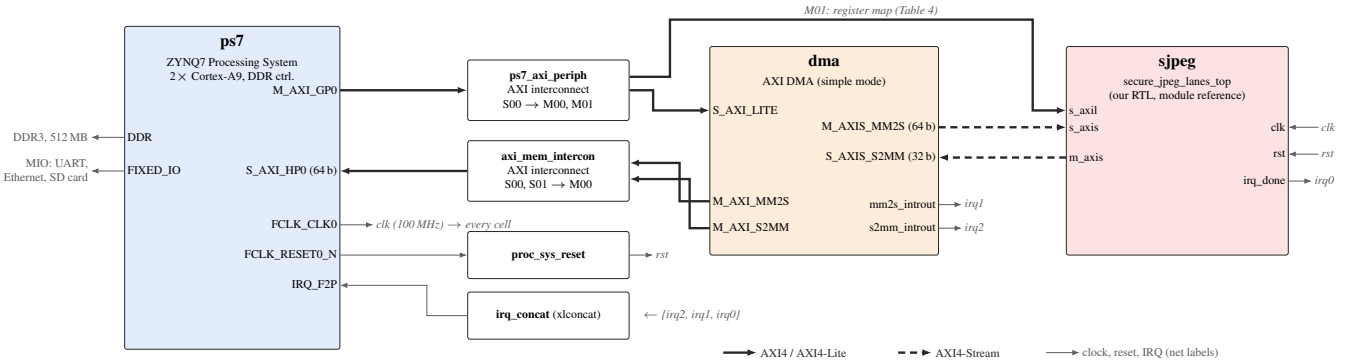


FIGURE 7. Block design of the PYNQ-Z2 system as built by our script. The processing system clocks the fabric at 100 MHz (FCLK_CLK0), controls the DMA and the encoder over M_AXI_GP0 and an AXI interconnect, and gives the DMA access to DDR over the 64-bit high-performance port S_AXI_HP0. The DMA streams pixels into the encoder (64 bits, two pixels per beat) and scan words out of it (32 bits). Interrupts of the encoder and both DMA channels are concatenated into IRQ_F2P; the reset block drives the encoder's active-high synchronous reset.

MCU rows than lanes, and under random stalls of both DMA streams; for the lane encoder with one, two and four lanes, including E5, where the test also runs the ARM permutation and the receiver. A self-checking Verilog-2001 testbench encodes stored frames in every level and is bit-exact with the model both in Icarus Verilog and in Vivado's simulator (XSim 2026.1). The board driver, the bit-exact board gate and the live sender are tested against an emulation of the bitstream that answers the register map with the model's bytes.

Bug log. Every defect found, in the RTL, the model, the board scripts or the experiment scripts, received a regression test and an entry with its root cause and lesson. Table 5 lists those that shaped the design. Two patterns recur: bit-exact simulation cannot reveal timing failures, and experiment scripts were wrong more often than the RTL.

Results from data. Every number in this paper is produced by a script from stored results and inserted by a generator, never typed; a test fails if a table in the manuscript differs from what the data produce, or if the text uses a number that the data do not define.

VIII. HARDWARE RESULTS

A. CYCLES AND THROUGHPUT

In RTL simulation of a 160×128 frame, the cycle counts of E0–E4 differ by at most 2 cycles (Table 6), which is handshake noise rather than encryption work: as Proposition 3 predicts, encryption costs no cycles. Above the floor of 1.5 cycles per pixel, the only cost is filling the first MCU row: on five frame sizes the RTL follows (24), 1.64 cycles per pixel at 160×128 and 1.53 at 640×480 (Fig. 8).

Where the cycles go. Monitoring every handshake of the RTL pipeline over 9 cases (three images, three levels) shows the same cycles per pixel to within 0.001 for every image and level, and an entropy path (run-length coding, encryption, Huffman coding, packing) that waits for data at least 42 % of the time (Fig. 9). Faster designs must therefore widen the transform path, not the cipher, which is what the lanes do.

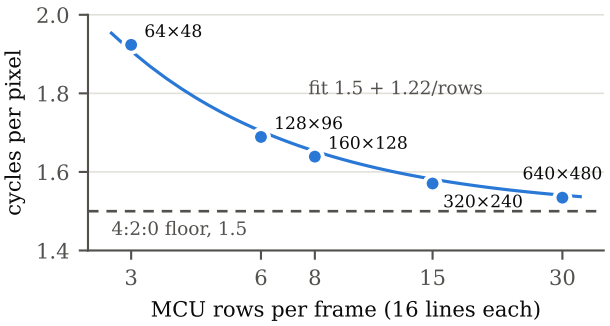


FIGURE 8. Cycles per pixel of the encoder RTL against frame height, measured in cycle-accurate simulation (dots), with the fitted law $1.5 + 1.22/r$ of (24) for r MCU rows.

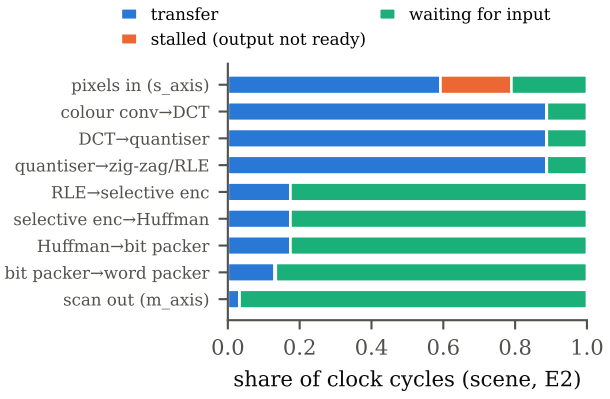


FIGURE 9. Where the clock cycles go: share of cycles in which each handshake of the encoder pipeline transfers data, is stalled by its consumer, or waits for its producer (RTL simulation, synthetic scene, E2). Everything after the run-length coder, including the cipher, idles most of the time.

Lanes. Measured at 100 MHz on 640×480 frames, bit-exact for every frame (Fig. 10), one, two and four lanes reach 66, 122 and 176 Mpixel/s (1.519, 0.817 and 0.569 cycles per pixel), against the bounds 67, 133 and 200 of (23). Four lanes give 2.7× the single pipeline, 572 frames/s at

TABLE 5. Defects that shaped the design (from our bug log).

Symptom	Root cause	Consequence
Two models gave different scans for one frame	sign rule, bit order, nonce layout and tag coverage had drifted	one contract, an equivalence test between the views
A level downgrade E2→E1 went undetected	the tag covered the scan only	26-byte authenticated prefix (17)
32-bit session identifiers	a repeat expected after about 2^{16} sessions	64-bit σ, high-word registers
Field top-bit flip as "sign"	changed the magnitude too ($5 \to -6$)	exact negation, Lemma 1
47 MHz after routing, all tests passing	a full DCT row summed in one clock	five-stage pipeline, post-route numbers in every claim
System missed timing, encoder alone met it	unconstrained input ports in the out-of-context run	I/O delays in every run
Board gate compared lanes with the wrong contract	reference chosen before the second contract existed	reference from the VERSION register
On the board the overlay loaded but the DMA hung	the block-design automation left the DMA's write master unconnected; simulation tests the IP, not the system around it	address-map check in the build, timeouts in the driver

TABLE 6. Encoder cycles per frame in RTL simulation (160x128, 100 MHz). The last column extrapolates to 640×480 at the same cycles per pixel.

Mode	Cycles	Cycles/pixel	Δ vs E0	fps (640×480)
E0	33519	1.637	+0	199
E1	33518	1.637	-1	199
E2	33521	1.637	+2	199
E3	33518	1.637	-1	199
E4	33519	1.637	+0	199

TABLE 7. Resources on the xc7z020 (53 200 LUT, 106 400 FF, 140 BRAM36, 220 DSP). Source: Vivado post-route, out of context, 100 MHz with I/O delays of 20 % of the period, WNS +0.554 ns.

Block	LUT	FF	BRAM36	DSP
Encoder (all)	5037 (9.5%)	2950 (2.8%)	24.5 (17.5%)	23 (10.4%)
colour conversion + line buffers	762	215	24	0
DCT and quantizer	806	69	0	22
zig-zag, Huffman, packing	720	323	0	0
keystream generator (Ascon)	950	851	0	0
tag unit (Ascon)	1345	1072	0	0
selective encryption	82	24	0	0
AXI-Lite registers	351	355	0	0

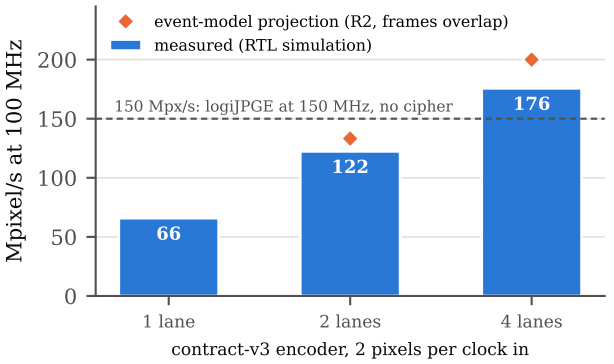


FIGURE 10. Throughput of the lane encoder, 640×480 frames at 100 MHz, measured in cycle-accurate simulation with every frame bit-exact against the golden model (bars), next to the event-model projection with overlapping frames (diamonds) and the published figure of a commercial encoder without cryptography [18] (dashed).

640×480 or 85 at 1080p, above the 150 Mpixel/s that the commercial encoder of [18] reports at 150 MHz without any cryptography, at two thirds of its clock. The gap to the bound has two causes: a frame starts only after the previous one is done, so every frame pays (24) once, and each segment adds a keystream restart and a byte flush. An event model that also lets the next frame's first MCU row arrive early projects 133 and 200 Mpixel/s (Fig. 11). The latency from the first pixel to the first scan word is 54 μs (17 lines of a 640×480 frame) for any number of lanes: the 16 lines of the first MCU row plus the pipeline. E5, with one restart per MCU, costs a few per cent of throughput in the lane encoder.

B. AREA, TIMING AND POWER

After place and route in Vivado the single-pipeline encoder meets 100 MHz on the xc7z020 (WNS +0.554 ns, out of context, I/O delays of 20 % of the period), i.e. 65 Mpixel/s or 212 frames/s at 640×480. Table 7 gives the resources (5,037 LUTs, 9.5 % of the device). The two Ascon instances take 46 % of the LUTs; the selective-encryption stage itself is tiny; all 24 BRAM36 are the converter's line buffers for 2048-pixel lines. With switching activity from simulating a real frame (SAIF), the encoder draws about 211 mW; the vectorless estimate is 223 mW. Two lanes take 10,069 LUTs, 50 BRAM36 and 46 DSPs (WNS +0.359 ns), four lanes 16,893 LUTs, 52 BRAM36 and 92 DSPs (WNS +0.293 ns, lines up to 1024 pixels). E5 adds no BRAM or DSP; with two lanes it draws 318 mW against 319 mW for E2. Both complete PYNQ-Z2 systems close timing at 100 MHz: the single pipeline with 6,933 LUTs (WNS +0.788 ns) and two lanes on a 64-bit pixel stream with 11,913 LUTs (WNS +0.228 ns); Vivado estimates 1.80 and 1.95 W for the complete systems, dominated by the processing system.

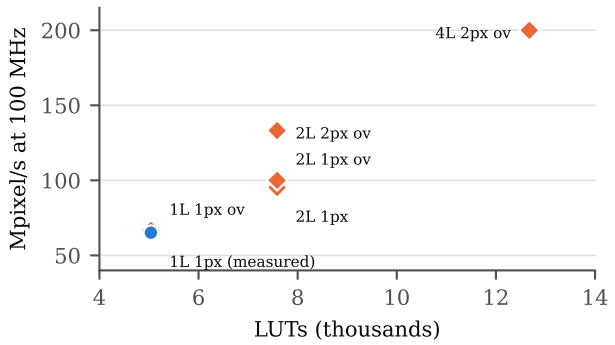


FIGURE 11. Projected throughput at 100 MHz against LUTs for 640×480 frames (event model calibrated on the measured RTL law). Labels: L lanes, pixels per clock at the input, "ov": the next frame overlaps the current one; the circle is the measured single pipeline.

TABLE 8. Comparison with published FPGA designs. Their numbers are as reported by each source; ours are post-route (Table 7) with throughput from the RTL cycle count of a 640×480 frame. Mpx/s: million pixels per second. Lanes (v3): throughput in RTL simulation at 100 MHz, area post-route. Kishore et al.: Stratix IV ALUTs and 18-bit multipliers, 256×256 grey images; Bağbaba and Örs report no throughput.

Design	Device	LUT	BRAM	DSP	MHz	Mpx/s	Crypto
Ours: encoder	Zynq-7020	5,037	24.5	23	100	65	sel. AEAD
Ours: one Ascon	Zynq-7020	861–969	0	0	100	–	AEAD core
Ours: 2 lanes (v3)	Zynq-7020	10,069	50.0	46	100	122	sel. AEAD
Ours: 4 lanes (v3)	Zynq-7020	16,893	52.0	92	100	176	sel. AEAD
logiJPGE [18]	Zynq-7020	3,240	16	30	150	150	none
CAST JPEG-E-S [19]	Artix-7	3,069	2	16	166	–	none
Khairallah [21]	Artix-7	1,031	0	0	200	–	MAC
Kurian [22]	Artix-7	5,335	32	0	100	–	AEAD
Kishore [9]	Stratix IV	2,250	–	47	100	11.1	scrambling
Bağbaba [16]	Virtex-6	2,130	–	9	87	–	TEA

C. COMPARISON

Table 8 sets the design against published FPGA work. Each of our Ascon cores (861–969 LUTs) is as small as the round-per-cycle Ascon-p cores of [21]; our JPEG datapath without the cipher (colour conversion, transform, quantizer and entropy coder in Table 7) uses fewer LUTs than the commercial encoders [18], [19], which however process one sample per clock at 150–166 MHz with less block RAM and no cryptography. Table 9 gives the figures of merit that hardware papers compare: 12.9, 12.1 and 10.4 Mpixel/s per thousand LUTs for one, two and four lanes, against 4.9 for the scrambling encoder of Kishore et al. [9] and 46.3 for the commercial encoder without cryptography [18]; and 3.2, 2.6 and 3.2 nJ per pixel. The lanes buy throughput, not efficiency: each lane carries its own transform and line buffers, so area grows with L while the energy per pixel stays roughly constant.

D. BOARD EVALUATION

Following the practice of Section II-F, the board evaluation on the PYNQ-Z2 has four parts, all scripted and rehearsed against an emulation of the bitstream: (A1) a bit-exact gate that encodes frames of two sizes in every level on the fabric and compares every byte and tag with the golden model; (A2) the encode time with registers and DMA, and the full frame

TABLE 9. Hardware figures of merit at 100 MHz, 640×480, E2: throughput from cycle-accurate RTL simulation (no DMA stalls), area and power post-route. Energy per pixel = on-chip power / throughput.

Design	LUT	Mpx/s	Mpx/s per kLUT	Gbit/s in	fps 1080p	Power (mW)	nJ/pixel
Ours, single pipeline (v2)	5,037	65	12.9	1.56	31	210 (SAIF)	3.22
Ours, 2 lanes	10,069	122	12.1	2.94	59	319 (SAIF)	2.61
Ours, 4 lanes	16,893	176	10.4	4.22	85	555 (vectorless)	3.16

TABLE 10. Two-lane E5 bitstream on the PYNQ-Z2 (100 MHz): every frame of every level E0–E5 compared byte for byte (scan and tag) with the golden model, and the time of a Python round trip on the ARM (pixel copy, cache flush, registers, DMA both ways), median of 50 frames; "frame" adds the JPEG headers (and for E5 the segment order) on the ARM. E0–E4 take the same time.

Size	Frames/level	Bit-exact	Encode (ms)	E2 frame, ms (fps)	E5 frame, ms (fps)
64×48	1	yes	3.3	4.7 (213.6)	5.7 (175.7)
320×240	10	yes	20.7	22.2 (45.1)	29.4 (34.0)
640×480	3	yes	90.7	92.2 (10.8)	117.2 (8.5)

time including the ARM's headers; (A3) the same frames in software on the board's Cortex-A9 (libjpeg-turbo, full-frame AES-GCM, our selective encryption), the fair baseline because that core has no AES instructions; and (A4) power from an inline meter, idle and under each workload, giving energy per frame. Parts A1 and A2 are done (Table 10). The two-lane E5 bitstream, rebuilt after one defect in the block design (the DMA's write master was left unconnected; Table 5 lists the lesson), is bit-exact with the golden model in every level E0–E5 at 64×48, 320×240 and 640×480: 84 frames, every scan byte and every tag. The board thus confirms Lemma 1 and the lane, E5 and tag logic on silicon, not only in simulation. With our Python driver a 640×480 frame takes 92 ms (10.8 frames/s; 45 at 320×240), and the times of E0–E4 differ by at most 0.09 ms: on silicon too, encryption costs no time. That time is dominated by the ARM, which converts and copies every pixel into the DMA buffer and flushes the cache, not by the fabric, which needs 2.5 ms per frame in RTL simulation; E5 adds 25 ms for splitting and reordering the segments on the ARM. [TODO: A2 fabric-only time (cycle counter), A3 ARM baseline, A4 measured power and energy per frame.]

IX. SECURITY EVALUATION

A. SETUP

We measure what a keyless observer can reconstruct. For every frame we take the better of two pictures: the protected stream decoded as is, and a replacement attack that substitutes fixed guesses for the encrypted values (absolute values for encrypted signs, the smallest value of the size category for encrypted amplitudes, zero DC differences) and re-encodes a valid JPEG. Leakage is the edge correlation (8) of this picture with the original; we also report SSIM [29]. Sources are a synthetic corridor scene; 5 clips that we analyse but do not reproduce (two film trailers, two talking-head videos and a handheld phone video, identified by hash); 2 openly licensed films of the Blender Foundation, the Sintel trailer and Tears

TABLE 11. Outline leakage: edge correlation between the original and the eavesdropper's best keyless reconstruction (decode or replacement attack; 0 = no outline), up to 60 frames per source; frames without edges are skipped. The first row is the fraction of payload bits encrypted. Jig.: share of neighbour pairs a greedy jigsaw solver restores under E5 (8 frames; Sintel 3); Oracle: edge correlation if every E5 block were put back in place, i.e. the outline E5 hides.

Source	E0	E1	E2	E3	E4	E5	Jig. (%)	Oracle
Enc. share	0.00	0.11	0.13	0.26	0.29	0.31		
Synthetic	1.00	0.38	0.36	0.38	0.38	0.00	2.3	0.37
Trailer A	0.99	0.47	0.30	0.32	0.28	0.01	1.4	0.46
Trailer B	0.99	0.56	0.34	0.33	0.32	0.01	1.3	0.41
Talk A	0.99	0.51	0.45	0.44	0.44	0.01	1.4	0.45
Talk B	0.99	0.49	0.40	0.42	0.40	0.01	1.7	0.47
Phone A	0.99	0.45	0.35	0.34	0.33	0.00	–	–
Sintel	0.99	0.35	0.29	0.31	0.28	-0.01	0.8	0.42
Tears of Steel	0.99	0.45	0.35	0.34	0.35	-0.01	1.3	0.41
Kodak	0.99	0.39	0.33	0.34	0.31	0.00	1.3	0.32

TABLE 12. Edge correlation of the eavesdropper's best keyless picture with 95 % bootstrap intervals over the frames of each source (last column: frames).

Source	E1	E2	E4	E5	n
Synthetic	0.38 [0.37, 0.38]	0.36 [0.36, 0.37]	0.38 [0.37, 0.38]	0.00 [-0.01, 0.00]	60
Trailer A	0.47 [0.46, 0.49]	0.30 [0.26, 0.34]	0.28 [0.24, 0.33]	0.01 [0.01, 0.02]	60
Trailer B	0.56 [0.54, 0.58]	0.34 [0.30, 0.38]	0.32 [0.28, 0.36]	0.01 [0.01, 0.02]	60
Talk A	0.51 [0.49, 0.52]	0.45 [0.42, 0.47]	0.44 [0.42, 0.46]	0.01 [0.00, 0.01]	60
Talk B	0.49 [0.47, 0.51]	0.40 [0.36, 0.43]	0.40 [0.38, 0.43]	0.01 [0.00, 0.01]	60
Phone A	0.45 [0.44, 0.46]	0.35 [0.32, 0.37]	0.33 [0.31, 0.36]	0.00 [0.00, 0.01]	60
Sintel	0.35 [0.30, 0.41]	0.29 [0.25, 0.33]	0.28 [0.25, 0.31]	-0.01 [-0.01, 0.00]	25
Tears of Steel	0.45 [0.42, 0.47]	0.35 [0.30, 0.39]	0.35 [0.30, 0.39]	-0.01 [-0.01, 0.00]	60
Kodak	0.39 [0.36, 0.42]	0.33 [0.27, 0.38]	0.31 [0.27, 0.35]	0.00 [0.00, 0.01]	24

of Steel (CC BY 3.0)[1]; and the 24 images of the Kodak set[2]. Video is resized to 320×240 and sampled every 150 frames; frames without edges are excluded.

Format compliance. The protected streams of all six levels decode without error in Pillow, OpenCV, scikit-image, FFmpeg (MJPEG), at the original resolution, as Lemma 1 requires.

B. OUTLINE LEAKAGE

Fig. 12 and Table 11 show the main effect. Under E1–E4 the eavesdropper sees a clear outline of the scene: edge correlation stays between 0.28 and 0.56 on the real sources, and encrypting amplitudes in addition to signs (E3, E4) does not reduce it, because the positions and size categories of the non-zero coefficients remain public (Lemma 1). The keyed macroblock permutation of E5 removes the outline (edge correlation between -0.007 and 0.014). In terms of cost, E2 encrypts 13.4 % of the payload bits, E4 29.5 %, and E5 only slightly more, 30.9 %, yet only E5 moves leakage to zero.

Statistical confidence. Table 12 and Fig. 13 give every leakage value with a 95 % percentile-bootstrap interval over the frames of its source. The intervals of E1–E4 are narrow and far from zero on all sources, so the outline leakage is not an effect of a few frames; the intervals of E5 contain zero or lie within a few hundredths of it.

[1]https://durian.blender.org, https://mango.blender.org
[2]http://r0k.us/graphics/kodak/

C. SVE-BENCH: PUBLISHED SCHEMES UNDER THE SAME ATTACKS

Leakage numbers from different papers are not comparable: each uses its own images, attack and metric. We therefore re-implemented four format-compliant schemes from their papers [3], [4], [6], [9], with our keystream so that only the scheme differs, and scored them and ours under the same four attacks on 175 frames of 9 sources (Table 13, Fig. 14):

- A1 decoding the protected stream as is;
- A2 the replacement attack;
- A3 non-zero counting: a picture built from the number of non-zero coefficients per block, which no field encryption hides [6];
- A4 a known-plaintext attacker that knows one frame in clear, recovers from it the permutation and the keystream bits of that frame, and applies them to the following frames.

At its strongest attack every published scheme leaks between 0.58 and 0.94. Those with fixed key material fall to the one known frame, among them Hirose et al. with a restart marker after every block (0.90), although it costs about as many bytes as E5 (+15 %). E1–E4 sit at the non-zero-counting ceiling of plain JPEG (0.79). E5, whose keystream and permutation change with every frame (Proposition 2), stays at or below 0.02 under every attack, in both its model and its hardware form; a test checks that the pictures an eavesdropper makes of the two forms are pixel-identical. The difference between E5 and Hirose et al. with one block per segment is not the mechanism, which is the same, but that our permutation is keyed per frame.

The metrics of image-encryption papers. Table 14 gives, for every scheme, the metrics that image-encryption papers report. They rank the schemes differently from Table 13: a scheme that scrambles symbols without hiding positions can score well on PSNR, entropy and NPCR while leaking its outline, because these metrics measure how noisy the picture looks, not what an attacker can extract from it.

D. JIGSAW SOLVERS AND A CHOSEN-PLAINTEXT ATTACK

A permutation of blocks invites a puzzle solver. Our greedy region-growing solver in the style of [14] restores most neighbour relations on unencrypted shuffled tiles. On E5 replacement-attack tiles it places at most 1.7 % of neighbour pairs correctly, and the solved picture has no outline (Table 11). The structure is not destroyed, only hidden: an oracle that knew the permutation would recover edge correlations of 0.32–0.47. A minimum-spanning-tree solver with Mahalanobis gradient compatibility [15], which restores at least 0.84 of the neighbours on unencrypted tiles of six photos (greedy: 0.62), is also at chance on E5 (at most 2.5 % on nine sources, Fig. 15). The solvers fail because E5 also encrypts signs and amplitudes, so the borders of neighbouring tiles no longer match.

A chosen-plaintext attack [10] fails too: the attacker films a designed pattern whose per-block non-zero counts reveal

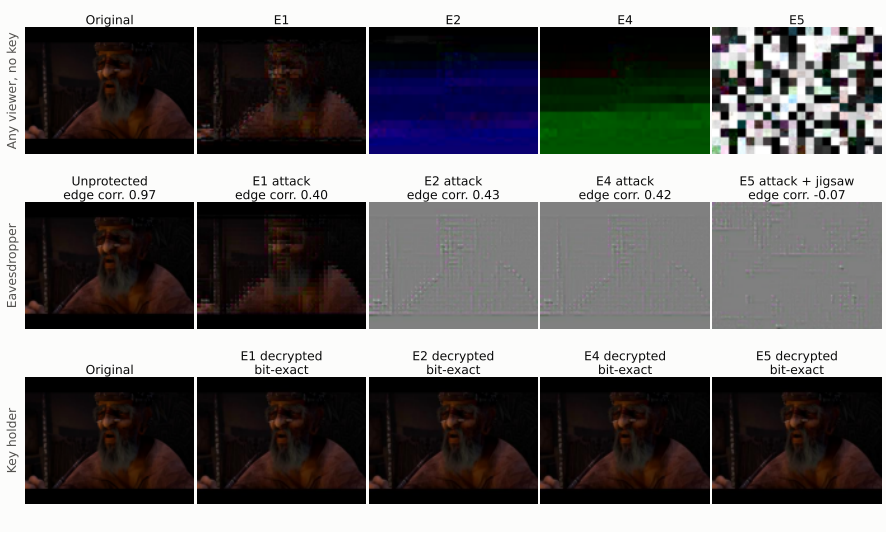


FIGURE 12. One frame of the Sintel trailer (© Blender Foundation, CC BY 3.0) per protection level. Top: what any viewer shows without the key. Middle: the eavesdropper's best reconstruction (replacement attack; for E5 followed by the jigsaw solver) with its edge correlation. Bottom: the key holder's output, bit-exact with the unprotected encoder.

TABLE 13. SVE-Bench: format-compliant selective encryption schemes under the same attacks (175 frames of 9 sources, 320×240, 768×512). Values are edge correlations with the original (0: nothing leaks; plain JPEG sets the ceiling of each attack). A4 knows one frame in clear: it recovers the permutation and the keystream bits from that frame and applies them to the following frames. Published schemes re-implemented from their papers with our keystream.

Scheme	Per-frame key	Auth.	Bytes (%)	A1 decode	A2 replacement	A3 NZCA	A4 known plaintext	Max
plain JPEG	–	no	+0	0.94	0.94	0.79	–	**0.94**
E1	yes	yes	+1	0.47	0.41	0.79	0.47	**0.79**
E2	yes	yes	+1	0.14	0.41	0.79	0.14	**0.79**
E3	yes	yes	+1	0.14	0.40	0.79	0.15	**0.79**
E4	yes	yes	+1	0.15	0.40	0.79	0.14	**0.79**
E5 (model)	yes	yes	+16	0.01	0.00	0.01	0.02	**0.02**
E5 (hardware form)	yes	yes	+16	0.01	0.00	0.01	0.02	**0.02**
Van Droogenbroeck [3]	no	no	+0	0.47	0.40	0.79	0.94	**0.94**
Kishore [9]	no	no	+0	-0.00	-0.00	0.00	0.78	**0.78**
Auer [4]	yes	no	+0	0.31	0.30	0.58	0.58	**0.58**
Hirose, RI = 1 [6]	no	no	+15	0.00	0.01	0.01	0.90	**0.90**
Hirose, RI = 4 [6]	no	no	+4	0.01	0.01	0.03	0.91	**0.91**

the permutation. The permutation comes from the frame nonce, so on all eight sources one frame's exact permutation places at most 0.4 % of the next frame's 300 tiles (chance) and leaves edge correlation $\leq$0.02, where a key-only shuffle would give 0.28–0.47.

E. SIGN-BIT RECOVERY BY OPTIMIZATION

The linear program of Lin et al. [5] treats the unknown signs as relaxed variables and minimizes the total variation of the picture subject to the known magnitudes and the pixel range. In their setting, where only the first three or six AC signs of a block are unknown, it recovers 88–93 % of the signs. On our levels it recovers 59 % of E1's signs (52–71 % per frame), where every AC sign is hidden, and 49 % of E2's (chance 50 %) on 29 frames of 9 source sets. It still sharpens E1's keyless picture (edge 0.41 against 0.33 for the replacement attack) and E2's and E3's by a few hundredths (Table 15): E1–E4 leak outlines however the hidden values are guessed.

F. LEARNED ATTACKERS

We trained a U-Net [31] to map the two keyless pictures (decoded and replacement attack) to the original and always tested it on clips it had not seen (Table 16). P1 and P2 train on 1000 frames of one clip, P3 uses only the open films, and P4 trains a wider network for 35 epochs on 3,417 frames of four sources and tests on 800 frames of three others. The network defeats sign-only encryption: under E1 it reaches an edge correlation of 0.69 and SSIM 0.79 (replacement attack: 0.40), so E1 is no protection against a trained attacker. Under E2 and E4 it adds little to the outline that replacement already reveals (edge correlation 0.37–0.43 against 0.39) but raises SSIM to about 0.37 by learning brightness and colour.

Controls for frame-level statistics. Table 17 and Fig. 16 compare each learned attacker with three pictures that use no ciphertext: the median training frame, the same frame recoloured with the mean and spread of colour that the E5 stream itself reveals, and the same frame recoloured with the

TABLE 14. The metrics image-encryption papers report, for every scheme of Table 13 (no-key picture; same frames). PSNR and SSIM of the luma against the original; entropy of the luma histogram (8: flat); correlation of adjacent pixels; NPCR/UACI between the pictures under two keys one bit apart, and between two frames one pixel apart. They were designed for full-image ciphers: a format-compliant scheme keeps the stream decodable by design, so read them next to Table 13.

Scheme	PSNR (dB)	SSIM	Entropy	Corr. h / v / d	NPCR / UACI key (%)	NPCR / UACI 1 px (%)
plain JPEG	42.7	0.97	5.93	0.90 / 0.90 / 0.86	– / –	0.1 / 0.0
E1	24.9	0.59	6.03	0.87 / 0.86 / 0.80	70.4 / 4.7	32.4 / 1.9
E2	11.0	0.29	5.09	0.85 / 0.87 / 0.82	86.8 / 46.1	32.2 / 11.3
E3	11.1	0.28	5.10	0.84 / 0.88 / 0.81	87.3 / 48.5	32.7 / 11.5
E4	10.2	0.28	4.97	0.82 / 0.86 / 0.79	86.6 / 48.2	32.4 / 11.9
E5 (model)	8.0	0.15	7.00	0.91 / 0.90 / 0.83	94.9 / 34.4	46.8 / 15.0
E5 (hardware form)	8.0	0.14	6.99	0.91 / 0.90 / 0.83	95.0 / 34.3	0.3 / 0.0
Van Droogenbroeck [3]	24.5	0.57	6.05	0.86 / 0.85 / 0.79	71.4 / 5.3	33.1 / 2.1
Kishore [9]	7.4	0.18	6.05	0.89 / 0.88 / 0.84	88.7 / 40.0	14.6 / 1.2
Auer [4]	16.9	0.40	6.24	0.63 / 0.61 / 0.49	84.6 / 21.7	75.9 / 17.7
Hirose, RI = 1 [6]	8.0	0.14	7.00	0.91 / 0.90 / 0.83	95.2 / 34.7	46.7 / 14.9
Hirose, RI = 4 [6]	7.6	0.18	6.93	0.96 / 0.92 / 0.89	94.1 / 37.1	45.7 / 16.3

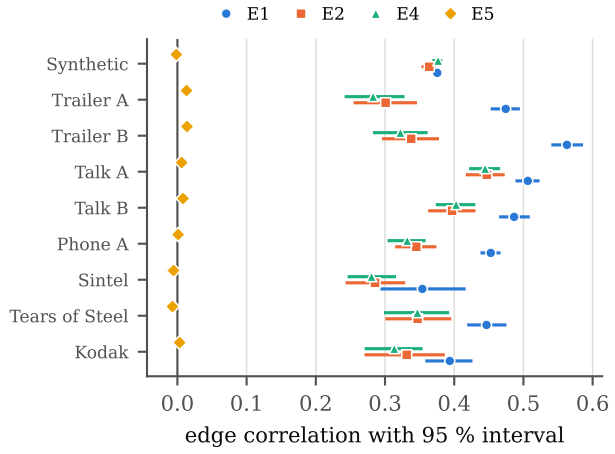


FIGURE 13. Edge correlation of the eavesdropper's best keyless picture per source and level, with 95 % bootstrap intervals over up to 60 frames. E5's intervals lie at zero on every source; E1–E4 leak an outline everywhere.

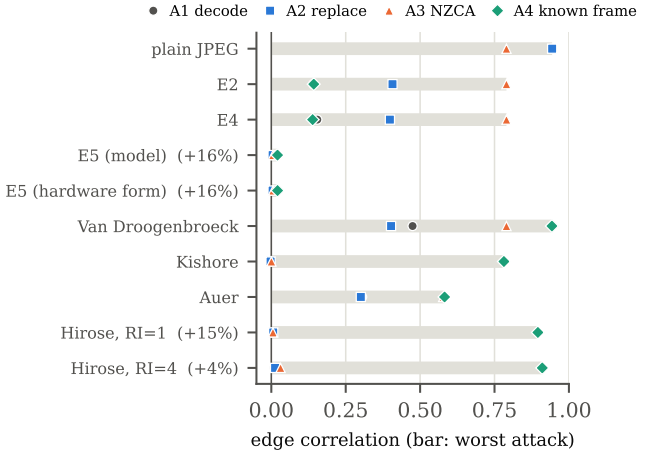


FIGURE 14. SVE-Bench in one view: edge correlation of every scheme under each attack, pooled over 9 sources; the bar ends at the worst attack. Byte cost over plain JPEG in brackets when above 2 %.

TABLE 15. Sign-bit recovery attack of Lin et al. [5] (LP relaxation, luma, 29 frames of 9 source sets). Top: their own setting on plain JPEG, where only the first U AC signs per block are unknown (check of our implementation). Bottom: our levels, attacked from the ciphertext alone. Signs: share of unknown AC signs recovered, weighted by magnitude (chance 50 %). Edge: edge correlation with the original; last column: the replacement attack for comparison.

Setting	Frames	Signs	SSIM	Edge	Repl. edge
Lin et al.'s setting, U=3	29	93 %	0.92	0.96	–
Lin et al.'s setting, U=6	29	88 %	0.87	0.92	–
E1	29	59 %	0.45	0.41	0.33
E2	29	49 %	0.20	0.35	0.32
E3	29	–	0.21	0.36	0.31
E4	29	–	0.16	0.30	0.31

TABLE 16. U-Net reconstruction, SSIM / edge corr. on unseen test clips. P1: Trailer A→B, P2: Talk A→B, P3: Sintel→Tears of Steel, P4: four sources→three others. Repl.: replacement attack, P4 test set. Last row: the median training frame, scored without looking at the stream.

Mode	Repl.	P1	P2	P3	P4
E1	0.57 / 0.40	0.72 / 0.53	0.70 / 0.66	0.55 / 0.41	0.79 / 0.69
E2	0.21 / 0.39	0.42 / 0.33	0.27 / 0.34	0.14 / 0.31	0.37 / 0.43
E4	0.22 / 0.38	0.43 / 0.34	0.28 / 0.32	0.13 / 0.30	0.36 / 0.37
E5	0.18 / 0.01	0.34 / -0.02	0.26 / 0.02	0.22 / 0.00	0.34 / 0.03
No key, no stream	–	–	–	0.21 / 0.04	0.25 / -0.00

true statistics of each test frame. Under E5 the U-Net of P4 scores SSIM 0.34 [0.33, 0.36], the oracle 0.36 [0.34, 0.37]; the revealed-statistics control reaches 0.26. SSIM rewards flat pictures in smooth regions, so a flat picture of each frame's true mean colour already scores 0.47; for this reason we read edge correlation, not SSIM, as the measure of recovered shape. Under E5 the network's edge correlation is 0.034 against 0.043 for the oracle: what it learns from the stream is brightness and colour, not the shape of the scene.

An adversarial attacker on public data. P5 trains the same U-Net for 35 epochs on 4,000 frames of Elephants Dream

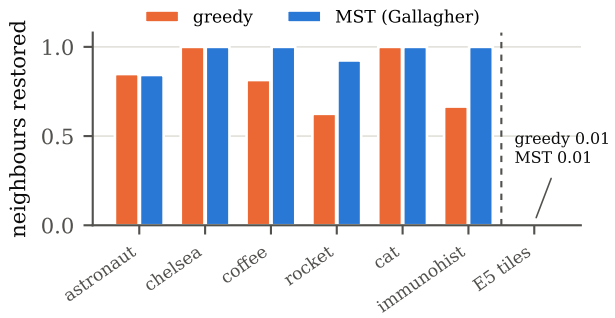


FIGURE 15. Neighbour pairs restored by the greedy and the minimum-spanning-tree solver on shuffled *unencrypted* tiles of standard test photos (left) and on E5 tiles averaged over all sources (right).

TABLE 17. Learned attackers with controls, SSIM/edge correlation on unseen test sources. R: replacement attack; N: U-Net. P3: Sintel→Tears of Steel; P4: four sources→three others; P5: public corpus with an adversarial loss→the Sintel trailer and the private clips. Controls use no ciphertext: the median training frame, recoloured with the frame statistics the E5 stream reveals or with the true ones.

	P3		P4		P5	
Mode	R	N	R	N	R	N
E1	0.45/0.36	0.55/0.41	0.57/0.40	0.79/0.69	0.61/0.47	0.83/0.74
E2	0.18/0.36	0.14/0.31	0.21/0.39	0.37/0.43	0.20/0.41	0.30/0.40
E4	0.19/0.35	0.13/0.30	0.22/0.38	0.36/0.37	0.21/0.40	0.31/0.41
E5	0.14/0.00	0.22/0.00	0.18/0.01	0.34/0.03	0.17/0.01	0.26/-0.01
No stream (median frame)		0.21/0.04		0.25/0.00		0.28/0.06
+ stats the stream reveals		0.22/0.04		0.26/0.00		0.26/0.06
+ true frame stats (oracle)		0.23/0.04		0.36/0.04		0.36/0.08

and Tears of Steel with an additional least-squares adversarial loss (weight 0.01) from a PatchGAN discriminator [32], and is tested on 800 frames of six sources it never saw. It is the strongest attacker on sign-only encryption (E1: edge correlation 0.74, SSIM 0.83), adds nothing to the outline that the replacement attack already shows under E2 and E4 (0.41 against 0.41), and recovers nothing under E5 (SSIM 0.26 below the no-stream control of 0.28; edge correlation -0.01).

G. RECOGNITION

Correlations measure whether a picture is in the right place; a person looking at the screen needs no such alignment. We therefore ran an off-the-shelf person detector, Faster R-CNN [33] with a ResNet-50 FPN backbone trained on COCO, on the original and on every picture an eavesdropper can make, at a score threshold of 0.5, and counted an original person as found if an attacker picture has a detection overlapping it by an IoU of at least 0.3. Fig. 17 and Table 18 show the result over 900 persons. On unprotected frames the detector finds 68–100 % of them. Sign-only encryption already defeats it on most sources (2 % pooled), but not on one talking-head clip (37 %); under E2 and E4 it finds at most 3 % on any source, and under E5 none. Detections anywhere in a frame, regardless of position, occur in at most 12 % of the frames with a person under E2–E5, about as often as false alarms on frames without one.

TABLE 18. Person detection (Faster R-CNN, COCO) in the eavesdropper's pictures: recall (%) of the persons found in the original frame, a detection in any attacker picture counting when it overlaps the original box (IoU $\geq$ 0.3). Sources with at least one person; 40 frames each (Kodak: 24 images).

Source	Persons	Plain	E1	E2	E4	E5/1	E5/32
Trailer A	69	75	0	0	0	0	0
Trailer B	63	81	0	0	0	0	0
Talk A	248	85	1	0	0	0	0
Talk B	38	100	37	3	3	0	0
Phone A	388	84	1	0	0	0	0
Sintel	34	68	0	0	0	0	0
Tears of Steel	29	97	0	0	0	0	0
Kodak	31	100	0	0	0	0	0

TABLE 19. E5 with N macroblocks per restart segment, mean over 8 sources. Overhead vs. E4 without markers; NZCA correlation; Succ.: segments chained to their true successor by a beam-search solver (chance).

N	Segments	Perm. bits	Overhead (%)	NZCA	Succ. (chance)
E4 (no shuffle)	–	–	–	0.81	–
1	300	2041	14.2	0.00	0.02 (0.00)
4	75	363	3.5	0.01	0.03 (0.01)
8	38	149	1.8	0.03	0.03 (0.03)
10	30	108	1.4	0.02	0.08 (0.03)
32	10	22	0.5	0.03	0.25 (0.11)

H. RESTART INTERVAL

E5 as specified costs 8–23 % over the same E4 coefficients at 320×240, as (19) predicts: a two-byte marker per macroblock, padding and restarted DC prediction. N macroblocks per segment reduce this to 3.5 % at N=4 and 0.5 % at N=32 (Table 19). Hirose et al. found that the non-zero counting attack reveals the scene unless the restart segments are permuted, and partly again when the restart interval is large [6]. Without the shuffle, our NZCA map correlates 0.72–0.88 with the original's block edge energy; with it, at most 0.08 for every N we tried. At N=32, however, only 10 segments remain, each an intact strip of 32 macroblocks, and a beam-search solver that chains segments by border continuity links 0.15–0.41 of them to their true successor on the real clips (chance 0.11), against at most 0.04 for $N \leq 8$. We keep N=1 and do not claim that longer intervals are safe.

I. STANDARD METRICS AND KEY SENSITIVITY

On all 8 sources (Table 20), key sensitivity holds: a one-bit key change makes E5 decryption differ in 77.5–99.5 % of the pixels (SSIM $\leq$0.22); in practice the receiver rejects such a frame because its tag fails. The other metrics assume a cipher that turns the image into noise: the 8×8 block structure survives, so adjacent-pixel correlation stays at 0.88–0.92 and entropy rises only under E5 (5.88 to 7.07 bits), and the 1-pixel NPCR (at most 47 %) measures diffusion, which a stream cipher with a fresh nonce per frame does not need: a one-pixel change of the plaintext under the same nonce never happens, because every frame has its own counter.

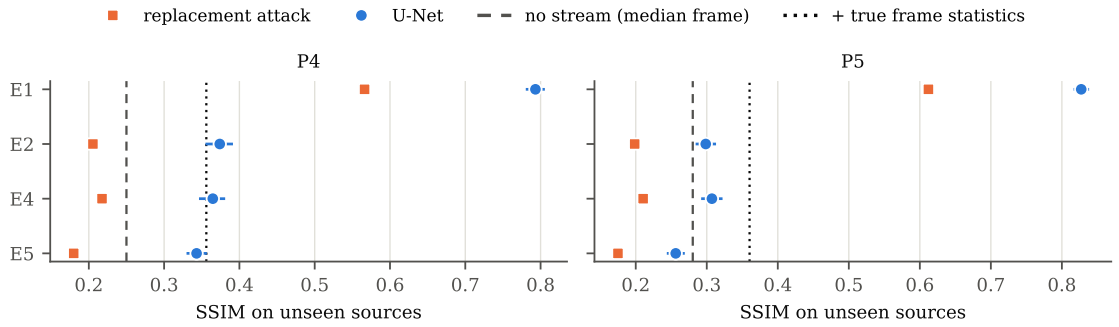


FIGURE 16. SSIM of the learned attackers (U-Net, with 95 % interval) and of the replacement attack on unseen sources, per level. Dashed: the median training frame, which uses no ciphertext; dotted: the same frame recoloured with each test frame's true colour statistics. P4: four sources to three others; P5: trained on public films with an adversarial loss, tested on six other sources.

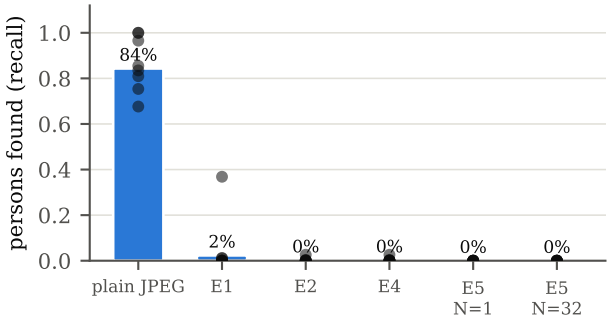


FIGURE 17. Share of the persons that a Faster R-CNN detector finds in the original frames and also finds, at the same place, in the eavesdropper's pictures (bars: pooled over eight sources; dots: per source).

TABLE 20. Standard image-encryption metrics, mean over 8 sources (5–6 frames each). Wrong key: the receiver's output with a key that differs in one bit, compared with the original (SSIM) and with the right decryption (NPCR/UACI, %). 1-pixel: NPCR between two ciphertexts of frames that differ in one pixel, same key and nonce. Ideal values for a full-image cipher: NPCR 99.6, UACI 33.5, entropy 8, adjacent correlation 0.

Level	Wrong-key SSIM	NPCR / UACI	1-pixel NPCR	Entropy	Adj. corr.
plain	–	–	–	5.88	0.93
E1	0.58	70.3 / 4.9	33.8	6.02	0.89
E2	0.27	89.8 / 46.3	35.5	5.40	0.92
E3	0.25	87.8 / 42.9	33.3	5.25	0.91
E4	0.24	88.8 / 47.5	34.5	5.14	0.91
E5	0.13	95.8 / 35.8	46.8	7.07	0.88

J. SECURITY SUMMARY

Against goal G1, E1 is broken by a trained network and E2–E4 leak the outline under every attack we tried, however many bits they encrypt; E5 removed the outline under all of them, including known plaintext, chosen plaintext, two puzzle solvers, sign recovery and two trained networks, at a byte cost of 8–23 %. What E5 leaks is the frame's colour statistics and its size; Section XII treats the latter.

X. PROTOCOL AND DEFENCE EVALUATION

Setup. Sender, network channel, receiver and logger run as separate processes that talk over UDP, so the evaluation exercises the real wire format, fragmentation and timing. The sender encodes 320×240 video at 10 frames/s with the golden model (the same bytes as the RTL), fragments every frame into datagrams of at most 1400 bytes and sends them; the channel can impose loss, delay, jitter and reordering and implements the attacker of Section IV. All runs used a 4-core cloud virtual machine; the same scripts run against the board (Section VIII-D).

TABLE 21. Reaction to each active attack (full suite, threshold policy, 3 runs; mean ± s.d. in seconds). "First rejection" is when the receiver first discards a frame; "mitigated" is when the sender has raised protection or rekeyed.

Attack	Mitigated	First rejection	Mitigated after
bit flips in the scan	3/3	0.09 ± 0.03	0.41 ± 0.02
mode downgrade in header	3/3	0.06 ± 0.02	0.44 ± 0.02
splice of two frames	3/3	0.13 ± 0.00	0.46 ± 0.01
forged frame injection	3/3	0.14 ± 0.06	0.44 ± 0.03
frame replay	3/3	0.07 ± 0.05	0.46 ± 0.03
frame delay	3/3	4.13 ± 0.04	1.46 ± 0.03
frame dropping	3/3	–	1.46 ± 0.03
junk flood	3/3	–	0.49 ± 0.04
feedback dropping	3/3	–	3.47 ± 0.05
forged feedback	3/3	–	0.46 ± 0.04
feedback replay	3/3	–	0.44 ± 0.01

Active attacks. The full scenario applies 11 attacks for six seconds each, 25 s apart: bit flips in the scan, a mode downgrade in the header, splicing of two frames, injection of a forged frame, replay, delay, dropping, a junk flood, and dropping, forging and replaying feedback. Table 21 gives the result over 3 runs. Every attack was mitigated in every run, on average 0.91 ± 0.02 s after it began, with 0/375 false alarms in calm windows and a median frame latency of 97 ms. The slowest case, dropped feedback, is caught only by the dead-man rule, after about 3.5 s: the 3 s timeout plus part of a decision window. With a trailer, a talking-head clip or Tears of Steel as the video source instead of the synthetic scene, all attacks were again mitigated (11/11, 11/11 and 11/11; means 0.89, 1.09 and 1.12 s, one run each) without false alarms.

The evidence decides the reaction time. The attacks of Table 21 are all mitigated, but Fig. 18 shows that they leave three different kinds of evidence, and the evidence, not the



FIGURE 18. Three attacks as the sender sees them (one full-suite run): frames per second from the receiver's authenticated heartbeats, and the protection level. A bit flip fails the tag at once; held-back frames produce no rejection while the attack lasts and return as a gap of lost frames; silenced feedback leaves the receiver's video intact but blinds the sender until the dead-man rule fires.

harm, sets how fast the defence can react. A bit flip is the easy case: the receiver rejects the first tampered frame after 0.09 s on average over three runs and the sender reaches E5 after 0.41 s. Holding frames back produces no rejection while it lasts, because nothing arrives at all. When the next frame gets through, the receiver counts the gap as lost frames; the held frames, released after the 4 s attack, still carry valid tags but are older than frames already shown, so they are discarded as late (25–30 per run, the first 4.1 s after the attack began), the verdict our replay window gives to benign reordering. The defence therefore never sees a delay: it reacts after 1.5 s to video that stops while heartbeats still arrive, which it treats as jamming. Silencing the feedback path is the mirror image. The receiver sees perfect video and rejects 0 frames, yet the sender, which learns about the link only through authenticated heartbeats, goes blind, and only the dead-man rule escalates, after 3.5 s. The stream stays at its floor level longest during this attack, which gives it the highest leakage of all attacks while active (0.14, the mean edge correlation of the active level). Integrity attacks are caught at the first frame; attacks by absence can only be caught by a timeout, so their reaction time is a design parameter, and shortening it trades against false alarms on a slow network.

Benign impairments. A second scenario applies only network problems: rising packet loss up to 6 %, delay up to 120 ms with jitter, and 5 % reordering. Over 3 runs the defence raised no false alarm (0/447 windows); median latency was 82 ms and the 95th percentile 278 ms. This needed loss to be judged over three windows and late frames to be told apart from replays.

A third of the frames lost, and no alarm. A frame of n fragments is lost when any fragment is, so with independent packet loss p

$$P_{\text{frame}} = 1 - (1-p)^n \approx np \quad (np \ll 1). \tag{25}$$

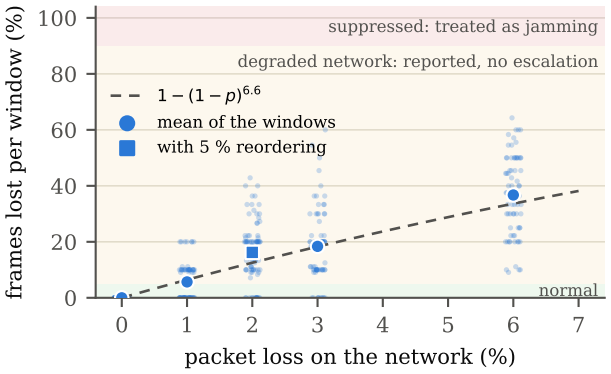


FIGURE 19. Frame loss against packet loss in the benign scenario (3 runs; small dots: one-second windows, large: means). The dashed curve is (25), fitted to the points without reordering. The shading is the policy's classification of frame loss.

Fig. 19 shows how badly a lossy network treats a fragmented video stream: at 6 % packet loss the receiver lost 37 % of the frames, 6 times the packet loss, and the points without reordering follow (25) with $n \approx 6.6$ fragments per frame. Even so, the defence did not escalate once, because loss between 5 and 90 % is reported as a degraded network; the highest threat score (20) of any benign window was 0.25, half the level at which Table 21 counts the sender as aware. The same rule is a limitation: jamming that lets more than a tenth of the frames through is indistinguishable from a bad network. Forward error correction across fragments would lower the curve, at a bitrate cost.

TABLE 22. Threshold rule vs. ε-greedy bandit on the learning scenario, 10 seeds each (mean $\pm$ s.d.). The last column counts the seeds in which the bandit was better (lower) than the threshold rule with the same seed.

Metric	Threshold	Bandit	Bandit better
Mitigation (s)	0.46 ± 0.02	1.34 ± 0.03	0/10
Leakage under attack	0.000 ± 0.001	0.054 ± 0.004	0/10
Encrypted share	0.188 ± 0.001	0.162 ± 0.001	10/10
Encrypted share, calm	0.135 ± 0.000	0.138 ± 0.001	0/10

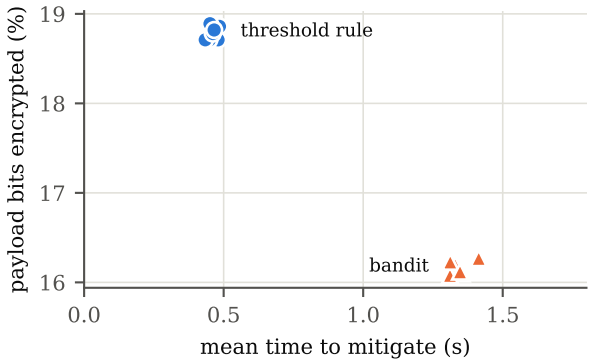


FIGURE 20. Encrypted share of the payload against mean time to mitigate, one point per seed and policy (10 paired seeds).

Threshold rule against bandit. The learning scenario repeats a bit-flip and a replay attack five times each, separated by calm periods. We ran both policies with 10 paired seeds (Table 22, Fig. 20). The result is a consistent trade-off rather than a winner. The bandit encrypted fewer bits in every seed (0.162 against 0.188 of the payload, 14 % less), but it reacted later (1.34 against 0.46 s) and leaked more while attacks were active (0.054 against almost zero), again in every seed. Neither policy raised a false alarm (0/6020). The reason is the loss (21): the bandit pays for encrypted bits in every calm window and for leakage only in the windows after an attack is visible, so it learns to sit lower and climb when evidence appears. We keep the threshold rule as the default; the bandit suits links where bandwidth matters more than the first second of an attack.

Cost relative to software. On the simulation host, our Python model needs 134 ms to encode a 640×480 frame and 151 ms including E2 selective encryption, whereas libjpeg-turbo takes 0.95 ms and full-stream AES-GCM about 0.01 ms. A desktop processor is the wrong baseline; what matters is the SoC's ARM core, which has no AES instructions (Section VIII-D).

XI. FURTHER EXPERIMENTS

A. BITRATE REGULATION

A sender on a shared link must follow the available bandwidth B_k. Treating the quality factor like the voltage of a motor drive, we compared fixed quality, bang-bang, integral, PI and feed-forward plus PI control (Fig. 21). JPEG maps quality q to the scale of the quantization tables as $5000/q$

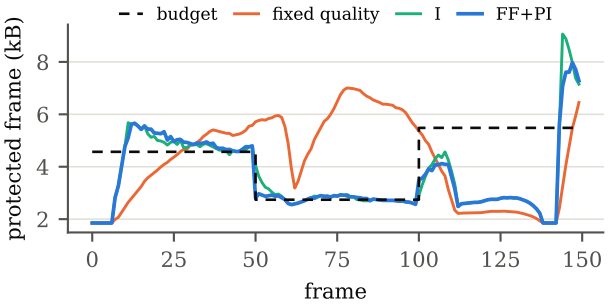


FIGURE 21. Bitrate regulation on the Sintel trailer: protected frame size under a budget that follows the link (100 %, 60 %, 120 % of the clip's median), with fixed quality, integral control and feed-forward plus PI.

below 50 and $200 - 2q$ above, so one quality step changes the bytes about four times more at low quality than at high quality; the controllers therefore act on $u = \ln(\text{scale})$, in which the plant is nearly linear, $\ln S_k \approx a - g\,u_k$, and on the logarithmic error

$$\varepsilon_k = \ln(B_k/S_k), \tag{26}$$

which is symmetric (twice too big counts as much as half too small). The PI law in velocity form, with a slew limit $|\Delta u| \le \Delta_{\max}$, is

$$\begin{aligned} \Delta u_k &= K_P(\varepsilon_k - \varepsilon_{k-1}) + K_I\varepsilon_k, \\ u_{k+1} &= u_k - \mathrm{clip}(\Delta u_k, \pm\Delta_{\max}), \end{aligned} \tag{27}$$

and the feed-forward term jumps by $\Delta u = -\ln(B_k/B_{k-1})/\hat{g}$ when the budget changes, with $\hat{g}$ estimated online by recursive least squares while the input is excited. After settling, fixed quality misses the budget by 29–144 % on seven real clips, integral control by 1–21 % and feed-forward plus PI by 2–30 %, which settles within 2 frames of a bandwidth drop (1.3 % on the synthetic scene). Scene cuts are disturbances that no causal controller anticipates. A controller tuned in the quality domain oscillated at low quality on one clip; acting on u keeps the loop gain nearly constant over the whole range.

B. CHANGE-DRIVEN CODING

Coding only the 16×16 blocks that changed, as a retina does, needs 22 % of the bytes on the fixed camera but 41–78 % on the real clips, and the number of coded blocks tracks motion (correlation 0.95 and 0.63–0.87) even when the change map is hidden (Fig. 22). Coding a constant 25 % of the blocks per frame, refreshing the oldest unchanged ones, and padding every frame hides both, at 46 % of the bytes and SSIM 0.945 on the fixed camera, but SSIM falls to 0.76–0.93 on edited clips.

C. KEYED BLOCK TRANSFORMS

Independent keyed flips per block defeat jigsaw solvers for block-scrambling ciphers. For the DCT, a flip is a pattern of sign changes, which E5 already encrypts: on seven sources

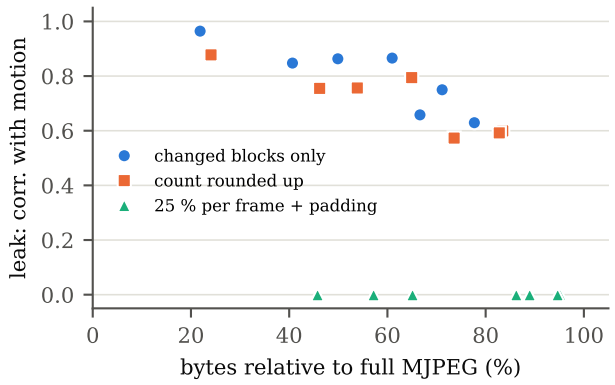


FIGURE 22. Change-driven coding: bytes against the leak of motion (the larger correlation of the per-frame block count or frame size with the true motion) for three policies, one point per source. Only coding a constant share of blocks and padding every frame hides motion.

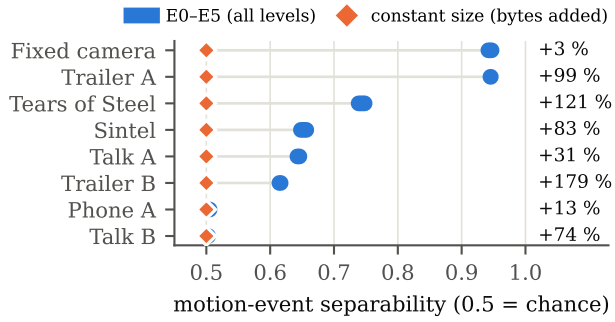


FIGURE 23. Motion events detected from frame sizes alone, for every protection level (bar: E0–E5 range) and with every frame padded to a constant size (diamond; bytes added on the right). Fixed camera: synthetic scene; the others are the real clips.

the jigsaw solver stays at chance (at most 2.3 % neighbours) with or without keyed flips, and even with every tile position given, SSIM changes by at most 0.008. E5's protection therefore rests on the block positions; an attacker who knew them would reach SSIM 0.05–0.38 on the real clips (0.55 on the synthetic scene).

XII. DISCUSSION AND LIMITATIONS

What no level hides. Encryption hides content, not length. Frame sizes reveal motion at every level alike (Fig. 23). They also identify a known video, as Schuster et al. showed for encrypted streaming [24]: an attacker who encodes candidate videos himself and correlates the size sequence picks the right one of 7 videos from ten frame sizes 98 % of the time under E1–E4 and 99 % under E5 (chance 14 %, Table 23). Padding to 1 KB steps lowers that to 40 % for +6 % bytes but not over 100 frames (97 %), padding to the running maximum to 23 % for +55 %, and only constant-size frames reach chance, at +81 % on these edited clips. Constant-size frames close the channel cheaply for a fixed camera (+3 %) but cost up to +179 % bytes on edited clips; padding to the largest

TABLE 23. Which of 7 videos is playing, identified from W consecutive encrypted frame sizes (after [24]), in % of 300 queries; chance 14 %. At 10 frames/s, $W=10$ is one second. Bytes: cost over the E2 stream.

Stream	Bytes	$W=5$	$W=10$	$W=30$	$W=100$
E1–E4	+0 %	76	98	100	100
E5	+12 %	69	99	100	100
pad to 1 KB	+6 %	21	40	85	97
pad to running max.	+55 %	18	23	31	46
pad to 10 s max.	+41 %	18	31	52	95
constant size	+81 %	14	13	19	14

frame so far gives a separability of 0.54 for +3 % on the fixed camera (0.54–0.78 on the real clips), while bitrate regulation lets up to 42 % of frames exceed a cap. A surveillance camera is the favourable case: its scene is fixed and padding is cheap.

Limitations. Five limitations bound our claims. First, the board confirms correctness but not yet speed or power: the encoder is bit-exact on the PYNQ-Z2, but our frame rate there is set by a Python driver on the ARM, the fabric-only time and the power are not yet measured on the board, and the power figures come from simulated switching activity; whether DDR feeds the four-lane encoder its 703 MB/s of pixels is also open. Second, sign and amplitude encryption leak outlines by construction (Lemma 1), however many bits are encrypted; only a change of block positions removes them, at a bitrate cost. Third, frame sizes leak at every level, as above. Fourth, our attackers are greedy or trained on a few clips; stronger puzzle solvers or attackers trained on large video corpora may extract more from E5, and a trained network already undoes sign-only encryption (E1). Fifth, the master key is pre-shared; a key exchange such as X25519 on the ARM core at session start, as DTLS-SRTP does for SRTP [1], would remove that assumption without touching the fabric. On the protocol side, the receiver cannot tell heavy benign loss from partial jamming, so only near-total suppression escalates, and the reaction time is bounded below by the one-second decision window.

Reproducibility and data. Every number in this paper is produced by a script from stored results and inserted into the text by a generator, never typed; a test fails if a table in the manuscript differs from what the data produce. The open sources (the Blender Foundation films and the Kodak images) can be downloaded and rerun; the private clips are identified by SHA-256 hash and never shown, and frames of them are never stored as images. The golden model, the RTL, the testbenches, the Vivado and board scripts, the experiment scripts and the scripts that rerun all GPU and long-CPU experiments in one cloud session are available in our repository.

Lessons. Three kinds of error recurred while building this system, and each changed how we work. Simulation-only success hid timing problems that only place and route exposed, so post-route numbers are now part of every hardware claim. Experiment scripts were wrong more often than the RTL: a set point that clips opening on black could not reach, an estimator that wound up when its input saturated, a con-

troller tuned at one operating point and unstable at another, and a test source that stamped the wall-clock time into every frame. Each was found because a result looked implausible on new data, and each now has a regression test. Finally, two claims had to be withdrawn when stronger evidence arrived: the E5 edge correlation of the U-Net is above that of the plain prior once more data is used, which the frame-statistics control explains, and the cost of hiding frame sizes is small only for fixed cameras.

Ethics. The clips with identifiable people are used only to measure what an eavesdropper could recover; no picture derived from them appears here, and none is stored.

XIII. CONCLUSION

Encrypting coefficient signs and amplitudes with Ascon inside a JPEG encoder costs no cycles, provably keeps every protection level playable, and, combined with a per-frame tag over metadata and scan, turns a plain MJPEG stream into an authenticated one. Field encryption alone, however, leaves the outline of the scene to anyone who counts non-zero coefficients, and published format-compliant schemes also fall to one known frame because their key material does not change. A keyed permutation of macroblocks that is drawn anew for every frame removes the outline under every attack we could build, and with restart segments it fits in hardware without a frame buffer: the fabric codes and authenticates the segments, the processor only reorders them. Parallel lanes over the same segments raise throughput to 176 Mpixel/s at 100 MHz, and a closed feedback loop answers active network attacks within about a second in simulation. No level hides frame sizes, which identify a known video and reveal motion; constant-size frames close that channel at a cost that is small only for fixed cameras. Next are the fabric-only time and power on the board, a driver that keeps the ARM off the pixel path, size padding in hardware, a shared Ascon core across lanes, a 125 MHz clock, key derivation with the standard's extendable-output function, and a key exchange at session start.

ACKNOWLEDGMENT

We thank Dr. Chetana G for her guidance. [TODO: AI-assistance disclosure as required by IEEE (see paper/README.md): name the tools and what they were used for, e.g. code generation, test scaffolding and language editing.]

REFERENCES

[1] M. Baugher, D. McGrew, M. Naslund, E. Carrara, and K. Norrman, "The secure real-time transport protocol (SRTP)," IETF, RFC 3711, Mar. 2004.

[2] A. Massoudi, F. Lefebvre, C. De Vleeschouwer, B. Macq, and J.-J. Quisquater, "Overview on selective encryption of image and video: Challenges and perspectives," EURASIP Journal on Information Security, vol. 2008, p. 179290, 2008.

[3] M. Van Droogenbroeck and R. Benedett, "Techniques for a selective encryption of uncompressed and compressed images," in Proc. Advanced Concepts for Intelligent Vision Systems (ACIVS), Ghent, Belgium, Sep. 2002, pp. 90–97.

[4] S. Auer, A. Bliem, D. Engel, and A. Unterweger, "Bitstream-based JPEG encryption in real-time," International Journal of Digital Crime and Forensics, vol. 5, no. 3, pp. 1–14, 2013.

[5] R. Lin, S. Liu, J. Jiang, S. Li, C. Li, and C.-C. J. Kuo, "Recovering sign bits of DCT coefficients in digital images as an optimization problem," Journal of Visual Communication and Image Representation, p. 104045, 2024.

[6] M. Hirose, S. Imaizumi, and H. Kiya, "On the security of bitstream-level JPEG encryption with restart markers," in Proc. Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), 2024, arXiv:2410.06522.

[7] M. S. Turan, K. A. McKay, D. Chang, J. Kang, and J. Kelsey, "Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions," National Institute of Standards and Technology, NIST Special Publication 800-232, Aug. 2025.

[8] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "Ascon v1.2: Lightweight authenticated encryption and hashing," Journal of Cryptology, vol. 34, no. 3, p. 33, 2021.

[9] B. Kishore, B. K. Shreyamsha Kumar, and C. R. Patil, "FPGA based simple and fast JPEG encryptor," Journal of Real-Time Image Processing, vol. 10, no. 3, pp. 551–559, 2015.

[10] S. Li, Y. Guo, J. Huang, and R. Gao, "Fast attack algorithm for JPEG image encryption with block position shuffle," Journal of Shanghai Jiaotong University (Science), vol. 30, no. 5, pp. 976–987, 2025.

[11] W. Puech and J. M. Rodrigues, "Crypto-compression of medical images by selective encryption of DCT," in Proc. 13th European Signal Processing Conference (EUSIPCO), Antalya, Turkey, 2005.

[12] B. Boyadjis, C. Bergeron, B. Pesquet-Popescu, and F. Dufaux, "Extended selective encryption of H.264/AVC (CABAC)- and HEVC-encoded video streams," IEEE Transactions on Circuits and Systems for Video Technology, vol. 27, no. 4, pp. 892–906, 2017.

[13] W. Sirichotedumrong and H. Kiya, "Visual security evaluation of learnable image encryption methods against ciphertext-only attacks," in Proc. Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC), 2020, arXiv:2010.06198.

[14] D. Pomeranz, M. Shemesh, and O. Ben-Shahar, "A fully automated greedy square jigsaw puzzle solver," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2011, pp. 9–16.

[15] A. C. Gallagher, "Jigsaw puzzles with pieces of unknown orientation," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2012, pp. 382–389.

[16] A. Ç. Bağbaba and B. Örs, "Hardware implementation of novel image compression-encryption system on a FPGA," in Proc. 9th International Conference on Electrical and Electronics Engineering (ELECO), Bursa, Turkey, 2015, pp. 1159–1163.

[17] H. Qiu, J. Lian, X. Wang, and J. Liu, "Securing satellite communications: Real-time video encryption scheme on satellite payloads," 2025, arXiv:2503.16287.

[18] Xylon d.o.o., "logiJPGE motion JPEG encoder data sheet, v1.0," https://www.logicbricks.com/Documentation/Datasheets/IP/logiJPGE_hds.pdf, 2020.

[19] CAST, Inc., "JPEG-E-S baseline JPEG encoder, product brief," https://www.cast-inc.com/pdfs/cast_jpeg-e-s-brief-amd.pdf, accessed 14 Oct. 2026.

[20] H. Groß, E. Wenger, C. Dobraunig, and C. Ehrenhöfer, "Ascon hardware implementations and side-channel evaluation," Microprocessors and Microsystems, vol. 52, pp. 470–479, 2017.

[21] M. Khairallah and S. Yadhunathan, "FPGA implementations of message authentication codes based on Ascon-p," in NIST Lightweight Cryptography Workshop, 2023.

[22] M. Gladis Kurian and Y. Chen, "Ascon on FPGA: Post-quantum safe authenticated encryption with replay protection for IoT," Electronics, vol. 14, no. 13, p. 2668, 2025.

[23] C. Wampler, S. Uluagac, and R. Beyah, "Information leakage in encrypted IP video traffic," in Proc. IEEE Global Communications Conference (GLOBECOM), 2015.

[24] R. Schuster, V. Shmatikov, and E. Tromer, "Beauty and the burst: Remote identification of encrypted video streams," in Proc. 26th USENIX Security Symposium, 2017, pp. 1357–1374.

[25] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the smart home private with smart(er) IoT traffic shaping," Proceedings on Privacy Enhancing Technologies, vol. 2019, no. 3, pp. 128–148, 2019.

[26] F. Farahmand, E. Homsirikamol, and K. Gaj, "A Zynq-based testbed for the experimental benchmarking of algorithms competing in cryptographic

contests," in Proc. International Conference on ReConFigurable Computing and FPGAs (ReConFig), Dec. 2016.
[27] ITU-T, "Information technology – digital compression and coding of continuous-tone still images – requirements and guidelines," International Telecommunication Union, Recommendation T.81, 1992.
[28] G. K. Wallace, "The JPEG still picture compression standard," IEEE Transactions on Consumer Electronics, vol. 38, no. 1, pp. xviii–xxxiv, 1992.
[29] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600–612, 2004.
[30] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.
[31] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI), ser. LNCS, vol. 9351, 2015, pp. 234–241.
[32] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros, "Image-to-image translation with conditional adversarial networks," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 1125–1134.
[33] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015, pp. 91–99.

Aravind R [TODO: biography and photo (1 in × 1.25 in), as the IEEE Access template requires]

Suhas V J [TODO: biography and photo]

Rishab H R [TODO: biography and photo]

Samarth B N [TODO: biography and photo]

Vijaysurya B S [TODO: biography and photo]

Chetana G [TODO: biography and photo]